

Optimization and Thermodynamics

From convergent attractors to algorithmic entropy, and what physics charges an embedded agent for steering the world

Version 2 (revised exposition)

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

A useful way to characterize powerful agents is that they reliably steer the world into a narrow region of outcome space, a region that would be extremely unlikely to arise under any random process. These notes develop that picture from first principles, in three stages. First, we make “steering into a narrow region” precise. We characterize an optimizer behaviorally, as the *cause* of a convergent attractor: a physical thing whose presence drives the system it acts on, from a broad range of initial conditions and despite perturbations, into a narrow target set, so that conditioning on the optimizer collapses one’s uncertainty about where that system ends up. We argue that even an observer who cares only about prediction has an objective, information-theoretic reason to single out such optimizers, which makes optimization an observer-independent feature of the world rather than a stance projected onto it. Translated into information theory, optimization is local entropy reduction in the system being acted on. Second, we ask what physics says about such processes. The reversibility of microscopic dynamics, together with the second law of thermodynamics, forbids global entropy reduction, so an embedded optimizer must balance its books. We derive the second law from reversibility in a minimal Markov-chain setting, classify the three ways an agent can reduce a subsystem’s entropy while respecting information conservation, and quantify the third of those ways with the Touchette–Lloyd theorem (entropy reduction beyond a blind baseline must be paid for in mutual information between the agent and the environment), developed in an appendix. A self-contained toy thermodynamics built out of biased coins (Wentworth’s generalized heat engine), in which every step of the bookkeeping is elementary, is developed in an appendix. Third, we confront a conceptual gap. The standard Gibbs–Shannon entropy is defined relative to a subjective probability distribution that the formalism treats as exogenous; this makes “capacity to optimize” observer-dependent and leaves the classical tools inapplicable to the far-from-equilibrium, information-bearing states that agents are made of. Algorithmic thermodynamics, due to Ebtakar and Hutter, replaces the subjective distribution with Kolmogorov complexity, yielding a second law that applies to individual physical states. Under this reframing, an embedded agent’s probabilistic knowledge becomes an endogenous, physically encoded quantity: the algorithmic mutual information between the agent’s memory and the environment, which is precisely the resource the agent spends when it optimizes. Agents that know more about the world can do more to it, and the exchange rate is set by thermodynamics.

Contents

1	Introduction: why thermodynamics belongs in agent foundations	3
2	A self-contained toolkit: probability, entropy, information	4
2.1	Random variables and notation	4
2.2	Entropy	4
2.3	The coding interpretation	5

2.4	Joint and conditional entropy, and mutual information	5
2.5	Divergence and two workhorse inequalities	6
3	What is optimization?	6
3.1	Two everyday notions, and the question that links them	6
3.2	Optimizers and convergent attractors	6
3.3	Optimization as entropy reduction	7
3.4	Why a predictor should attend to optimizers	7
3.5	Optimization is an objective feature, not observer-relative	8
4	Reversibility and the second law	9
4.1	Microscopic physics is reversible	9
4.2	Why the funneling picture cannot hold globally	9
4.3	Coarse-graining: where probability enters a deterministic world	10
4.4	The second law of thermodynamics	10
4.5	What the second law does and does not forbid	11
5	Three types of optimization under information conservation	12
5.1	The bookkeeping problem	12
5.2	Type 1: dump waste heat into the environment	12
5.3	Type 2: absorb the entropy into the agent's memory (measurement)	12
5.4	Type 3: spend mutual information you already have	13
5.5	The demon reframed, and the recap	13
6	The trouble with subjective entropy	14
6.1	Entropy is supposed to measure the capacity to optimize	14
6.2	The exogenous distribution	14
6.3	Three ways the ensemble picture fails	15
6.4	The demon's capacity, and why this is the central case for agent foundations	15
7	Algorithmic thermodynamics	16
7.1	Kolmogorov complexity	16
7.2	Why K deserves to be called entropy	17
7.3	The algorithmic second law	18
7.4	Maxwell's demon, exactly	19
8	Knowledge as a physical resource: optimization for embedded agents	20
8.1	From exogenous to endogenous knowledge	21
8.2	The three types, sharpened by universal computation	22
8.3	Dissolving the subjectivity	22
8.4	Back to optimizers	22
9	Takeaways	23
A	A thermodynamics of biased coins: the generalized heat engine	24
A.1	The designer's viewpoint	24
A.2	The setup: coins, transformations, and two conservation laws	24
A.3	Extracting work is a compression problem	25
A.4	No work from a single heat bath	25
A.5	Work from two heat baths at different temperatures	26
A.6	What the toy world teaches	26

B Steering costs information: the Touchette–Lloyd theorem	27
B.1 From optimization to modeling: the question	27
B.2 The setup: environments, actions, policies	27
B.3 Blind and sighted policies	28
B.4 A worked example: the guessing game, played blind	28
B.5 The same game, played sighted	29
B.6 Mutual information measures sightedness	29
B.7 The theorem	29
B.8 What the theorem does not say	30

1 Introduction: why thermodynamics belongs in agent foundations

Agent foundations is the search for *robust concepts* (sometimes called “true names”) for things like optimization, goals, world models, and embeddedness. A robust concept is one that keeps its meaning under extreme optimization pressure, and for agents far more capable than any we have built. This document goes after the true name of *optimization*, and it takes the descriptive route. Rather than start inside an idealized rational agent, with its beliefs and preferences and expected utility, we start from the physical world and ask one question: what kind of process can reliably steer the world into a narrow target, and what does physics charge it for doing so?

Thermodynamics answers the second half of that question, for two reasons. Here they are up front; the rest of the document unpacks them.

Optimization is local entropy reduction. An optimizer concentrates probability mass: out of the many configurations the world could be in, it funnels a broad set down to a few. But concentrating probability mass just is reducing uncertainty, and reducing uncertainty just is reducing entropy (Section 2 makes this exact). This is not an analogy. It puts the full machinery of entropy (the second law, fluctuation theorems, the cost of erasing a bit) directly in charge of optimization, and the modern version of that machinery, *stochastic thermodynamics*, holds arbitrarily far from equilibrium. So physics constrains the shape of any optimizer you could actually build.

Embedded agents pay for knowledge in physical currency. Picture two kinds of agent. A *dualistic* agent sits outside the world it acts on, like a player at a console: clean inputs, clean outputs, and “what it knows” a free parameter, a distribution we write down by hand. An *embedded* agent is part of the world it acts on, so its knowledge is not free. It has to be stored somewhere physical, in a brain or a memory register made of the same atoms as everything else. One of the main conclusions of these notes is that once you define entropy correctly (algorithmically, as a property of an individual state rather than of a subjective ensemble), an agent’s knowledge of its environment turns into an objective physical quantity, the mutual information between its memory and the environment, and that quantity is exactly the budget it has to spend on optimization.

A word on what is new here, and for whom. If you have a physics background, the value is mostly translation rather than new theorems: recasting familiar thermodynamic facts in information-theoretic language ties them to agent-foundations notions like world models, optimization power, and the limits on embedded agents. If you do not, the notes are self-contained. Every thermodynamic idea is defined here in information-theoretic terms, no statistical mechanics is assumed, and the only prerequisite is comfort with elementary discrete probability.

Plan of the notes. Section 2 builds the probabilistic and information-theoretic toolkit from scratch: entropy, mutual information, divergence, and the coding interpretations that make these quantities meaningful. Section 3 asks what optimization is: it characterizes an optimizer behaviorally, as a physical thing that drives the system it acts on, from a broad range of initial conditions and robustly to perturbation, into a narrow target (a convergent attractor), translates this into entropy reduction, and argues that any observer who cares about prediction has an objective, information-theoretic reason to attend to optimizers, whoever is watching. Section 4 confronts this picture with the reversibility of microscopic physics and derives the second law of thermodynamics, with full proof, in a minimal coarse-grained setting. Section 5 then classifies

the three ways an embedded agent can reduce a subsystem’s entropy while respecting global information conservation: dumping it into the environment, absorbing it into memory by measurement, or spending pre-existing mutual information with the subsystem. Section 6 exposes a conceptual problem at the heart of the standard formalism: entropy, as usually defined, depends on a subjective probability distribution supplied from outside the physics. Section 7 develops the resolution, algorithmic thermodynamics: Kolmogorov complexity as entropy, the algorithmic second law, and an exact analysis of Maxwell’s demon, with the emphasis throughout on how an objective notion of entropy dissolves the subjectivity problem. Section 8 assembles the synthesis for embedded agency: knowledge as an endogenous physical resource. Section 9 collects the takeaways. Appendix A develops, in parallel to the main line, a self-contained toy thermodynamics built out of biased coins (Wentworth’s generalized heat engine), in which the analogues of heat, work, energy conservation, and the impossibility of perpetual motion can all be verified by hand; the main text points to it wherever a concrete blind-policy example helps. Appendix B develops the Touchette–Lloyd theorem, which makes the third channel quantitative: the entropy reduction an agent can achieve beyond a blind baseline is bounded by the mutual information between its action and the environment, with fully worked examples.

2 A self-contained toolkit: probability, entropy, information

This section develops everything we need from information theory. If you already know Shannon entropy and mutual information, skim it, but note the two things we lean on later: the coding interpretation of Section 2.3, which recurs constantly, and the log-sum inequality (Lemma 2.9), which powers our proof of the second law.

2.1 Random variables and notation

Throughout, capital letters X, Y, Z denote random variables taking values in countable (finite or countably infinite) sets, and lowercase letters x, y, z denote particular values. We write $\mu(x)$ for the probability that $X = x$ when the distribution is called μ , and we freely say “the distribution of X ” or “the ensemble” for μ . A joint random variable (X, Y) has a joint distribution, and the conditional probability of y given x is written $P(y | x)$. All logarithms in these notes are base 2, so that information is measured in bits; the natural-log versions of every formula differ only by a factor of $\ln 2$.

2.2 Entropy

Definition 2.1 (Shannon entropy). The *Shannon entropy* of a random variable X with distribution μ is

$$H(X) := \sum_x \mu(x) \log \frac{1}{\mu(x)},$$

with the convention that terms with $\mu(x) = 0$ contribute zero. We also write $H(\mu)$ for the same quantity when we want to emphasize the distribution rather than the variable.

Entropy measures uncertainty: how much you do not know about the value of X before you observe it. Some examples will calibrate the scale.

Example 2.2 (Calibrating entropy). A fair coin has entropy $\frac{1}{2} \log 2 + \frac{1}{2} \log 2 = 1$ bit. A deterministic variable (one whose value is certain) has entropy 0 bits. A uniform distribution over all binary strings of length n has entropy n bits, and n bits is the maximum possible entropy for a variable with 2^n possible values: uniform distributions are the most uncertain ones. A *biased* coin that lands heads with probability p has entropy $h(p) := p \log \frac{1}{p} + (1 - p) \log \frac{1}{1-p}$. Two values of this function will matter later in these notes: $h(0.2) \approx 0.72$ bits and $h(0.1) \approx 0.47$ bits. A coin biased toward one outcome is more predictable than a fair coin, so it carries less than one bit of uncertainty.

2.3 The coding interpretation

We will lean on one reading of entropy more than any other: entropy as data compression. Suppose you have to transmit the value of X to a colleague in binary, and you want to send as few bits on average as you can. The trick is to spend short codewords on the likely values and long ones on the unlikely values. Shannon's source coding theorem makes this precise: there is a code (a prefix-free assignment of binary strings to outcomes) whose codeword for outcome x has length essentially $\log \frac{1}{\mu(x)}$ bits, and no code can achieve a smaller average length than the entropy. Therefore

$H(X)$ = the minimum average number of bits needed to describe a sample of X .

Example 2.3 (A small optimal code). Let X take four values with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$. Assign the codewords 0, 10, 110, 111. Each codeword for an outcome of probability $2^{-\ell}$ has length exactly $\ell = \log \frac{1}{\mu(x)}$, and the average length is $\frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75$ bits, which equals $H(X)$ exactly.

The quantity inside the expectation needs its own name, because Section 6 turns on the distinction between it and its average.

Definition 2.4 (Shannon codelength, also called stochastic entropy or surprisal). For an individual outcome x under distribution μ , the *Shannon codelength* is

$$\hat{H}(x, \mu) := \log \frac{1}{\mu(x)}.$$

It is the length of x 's codeword under the code optimized for μ , and the entropy is its mean: $H(\mu) = \langle \hat{H}(X, \mu) \rangle_{X \sim \mu}$.

Notice carefully what depends on what here. The codelength $\hat{H}(x, \mu)$ is a property of the *pair* (outcome, distribution): the same physical outcome x gets a short codelength under a distribution that expected it and a long one under a distribution that did not. So far there is no such thing as “the entropy of an individual outcome x ” on its own, with no distribution to read it against. Hold onto that; it becomes the crux of Section 6.

2.4 Joint and conditional entropy, and mutual information

Definition 2.5 (Conditional entropy). For jointly distributed X, Y , the *conditional entropy* of X given Y is

$$H(X | Y) := H(X, Y) - H(Y),$$

where $H(X, Y)$ is the entropy of the pair. Equivalently, $H(X | Y)$ is the average, over values y , of the entropy of the conditional distribution of X given $Y = y$. It measures the uncertainty that remains about X once Y is known.

The rearranged identity $H(X, Y) = H(Y) + H(X | Y)$ is called the *chain rule*, and it has a transparent coding reading: to describe the pair, first describe Y (costing $H(Y)$ bits on average), then describe X using a code adapted to the known value of Y (costing $H(X | Y)$ bits on average).

Definition 2.6 (Mutual information). The *mutual information* between X and Y is

$$I(X; Y) := H(X) + H(Y) - H(X, Y) = H(X) - H(X | Y) = H(Y) - H(Y | X).$$

Mutual information is the number of bits that knowing Y saves you, on average, when describing X ; by the symmetry of the definition, it is also the number of bits knowing X saves you about Y . It is zero exactly when X and Y are independent, and it is never negative (we prove this below). It is the standard measure of “how much one part of the world knows about another”, and that informal phrase will recur many times in these notes.

Example 2.7 (Mutual information, calibrated). Let $X = (B_1, B_2)$ consist of two independent fair bits, and let $Y = B_1$ be the first of them. Then $H(X) = 2$, and once you know Y only the second bit remains uncertain, so $H(X | Y) = 1$ and $I(X; Y) = 1$ bit: Y contains exactly one bit of information about X . If instead Y were an independent coin flip, $I(X; Y)$ would be 0; if Y were a full copy of X , $I(X; Y)$ would be 2 bits, the whole entropy of X .

2.5 Divergence and two workhorse inequalities

Definition 2.8 (Kullback–Leibler divergence). For two distributions μ, ν on the same countable set,

$$D(\mu \parallel \nu) := \sum_x \mu(x) \log \frac{\mu(x)}{\nu(x)}.$$

In coding terms, $D(\mu \parallel \nu)$ is the price of being wrong about the source: if the truth is μ but you compress using the code optimized for ν , you spend on average $H(\mu) + D(\mu \parallel \nu)$ bits instead of $H(\mu)$. The definition makes sense, and we will use it, even when the reference ν is an unnormalized nonnegative measure rather than a probability distribution.

Both of the inequalities we need follow from a single elementary fact about the convex function $t \mapsto t \log t$.

Lemma 2.9 (Log-sum inequality). *Let $a_1, a_2, \dots$ and $b_1, b_2, \dots$ be nonnegative reals with finite sums $a := \sum_i a_i$ and $b := \sum_i b_i > 0$. Then*

$$\sum_i a_i \log \frac{a_i}{b_i} \geq a \log \frac{a}{b},$$

with the conventions $0 \log \frac{0}{b} = 0$ and $a \log \frac{a}{0} = +\infty$ for $a > 0$.

Proof. The function $\varphi(t) = t \log t$ is convex on $[0, \infty)$. Apply Jensen’s inequality (for convex φ , the average of φ is at least φ of the average) with weights $w_i = b_i/b$ and points $t_i = a_i/b_i$:

$$\sum_i \frac{b_i}{b} \varphi\left(\frac{a_i}{b_i}\right) \geq \varphi\left(\sum_i \frac{b_i}{b} \cdot \frac{a_i}{b_i}\right) = \varphi\left(\frac{a}{b}\right).$$

Multiplying both sides by b and simplifying $b_i \cdot \frac{a_i}{b_i} \log \frac{a_i}{b_i} = a_i \log \frac{a_i}{b_i}$ gives the claim. $\square$

Taking all the a_i and b_i to be the probabilities of two distributions ($a = b = 1$) yields *Gibbs’ inequality*: $D(\mu \parallel \nu) \geq 0$, with equality only when $\mu = \nu$. Nonnegativity of mutual information follows because $I(X; Y) = D(\text{joint distribution} \parallel \text{product of marginals}) \geq 0$. The second consequence, which we prove when we need it in Section 4, is the *data processing inequality* for divergence: pushing two distributions through the same noisy channel can only bring them closer together.

With the toolkit in hand, we can turn to the actual subject.

3 What is optimization?

3.1 Two everyday notions, and the question that links them

The word “optimization” already means two different things, and the gap between them is the whole problem. In computer science, an optimization algorithm is a program that outputs the solution, or an approximation of it, to an optimization problem: an objective function to be minimized over some feasible region. A program that returns $x \approx \sqrt{2}$, the minimizer of $y = (x^2 - 2)^2$ on the real line, is optimizing in this sense. In operations research and engineering, optimization means something physically heftier: reworking a process or an artifact so it better fits a purpose, like reorganizing a factory to make more nails per unit cost. Optimizing a number inside a computer and optimizing a factory are obviously related, but what *is* the relation? What does a physical process have to do to earn the name optimization?

3.2 Optimizers and convergent attractors

Two things have to be kept apart. First, the *system being optimized*: some part of the world whose configuration we care about, and which, left alone, could end up all sorts of ways. Second, the *optimizer*: a separate physical thing whose presence drives that system reliably toward a narrow target. The optimizer is not the target, and it is not the funnelling; it is the *cause* of the funnelling. Here is the cleanest way to hold the idea: an optimizer is a physical thing such that, once you know it is there, your uncertainty about where the optimized system ends up collapses.

A courier delivering a package to a fixed address makes the two roles concrete. The package is the system being optimized; its location could be almost anywhere. The courier is the optimizer. Start the package off anywhere in the city, throw road closures and traffic in its way, and it still arrives, because the courier finds a route around each obstacle. Left alone the package goes nowhere in particular; the courier is what drives it to the address. In the language of dynamical systems, the courier makes the address a *convergent attractor* for the package: a configuration it is pulled toward from far away and returns to after being knocked aside. The attractor is a fact about the package’s behavior; the courier is what creates it.

Two features carry all the weight. The first is a *broad range of initial conditions*: the optimizer does not need the system to start anywhere special, because lots of different starting points all get driven to the same place. Someone who knows only that the package “started somewhere in the city” is very unsure where it began, but tell them a courier is delivering it and they become confident about where it ends up. The second is *robustness to perturbation*: the convergence survives disturbances thrown at it while the process is underway. This is what separates a real optimizer from a coincidence. A satellite coasting in orbit has nothing steering it; nudge it and it drifts onto a new path forever, with no home to return to. The courier, blocked by a closed road, takes a different one and still arrives. The whole difference is whether something is there to supply that restoring tendency.

One omission matters for everything that follows: calling something an optimizer says nothing about goals, preferences, or representations stored in a mind. A courier, a feedback circuit, a chess engine driving a game toward checkmate, a growing organism, a running optimization algorithm, and a team building a house are optimizers in exactly the same sense, each a physical thing whose presence drives some system into a narrow target and holds it there against perturbation. What unifies them is a structural fact about their effect on the world, not a mental attribute. The next three subsections turn this picture into a number, explain why even an observer with no stake in the target should care about it, and argue that optimizer-hood is therefore an objective feature of the world.

3.3 Optimization as entropy reduction

The convergent-attractor picture translates directly into the language of Section 2, and the translation is the bridge to physics.

Model the configuration of the optimized system as a random variable: X for where it starts, Y for where it ends. Before we account for any optimizer, X ranges over a broad set, so someone who knows only that the system starts somewhere in that set has high uncertainty $H(X)$. Now add the optimizer and condition on its presence. It drives the system into the narrow target no matter where the system started, so the uncertainty left about the final configuration, $H(Y)$, is small. The gap between the two is the *entropy reduction*

$$\Delta H := H(X) - H(Y),$$

the number of bits by which the optimizer has narrowed the system’s possibilities. A broad range of initial conditions makes $H(X)$ large; robustness is what keeps $H(X)$ large while the convergence still works; a narrow target makes $H(Y)$ small. A large ΔH means reliably hitting a target the system’s own unaided dynamics would essentially never reach, which is just the opening slogan made precise: a powerful optimizer reliably produces outcomes that would be wildly unlikely under any random process.

Remark 3.1 (What the translation keeps and what it drops). The summary ΔH deliberately forgets the *direction* of optimization: it records how narrow the achieved region is, not which region it is or whether anyone wanted it. This is less of a loss than it may appear. Wentworth has observed that any expected utility maximization problem can be decomposed into two parts: an entropy-minimization part, and a part that makes the world’s distribution resemble one particular target distribution (in his slogan, “utility maximization is description length minimization”). These notes are about the first part, which is where the laws of physics impose their constraints; the second part (which target, and why) is the business of the theory of goals and preferences, not of thermodynamics.

3.4 Why a predictor should attend to optimizers

Here is a reason to care about optimizers that assumes nothing about anyone’s goals. Suppose all you want to do is *predict* the future configuration of some part of the world. The claim is that, of all the things you

could study, optimizers are among the cheapest to know about.

Start with the ordinary case, no optimizer involved. To predict the final state Y of a physical system, you generally have to know its initial state X in detail and push it through the dynamics. That is expensive twice over. Specifying the initial condition costs about $H(X)$ bits, and for a system of any size $H(X)$ is enormous. Worse, the prediction decays as you look further ahead: chaos amplifies any error in your knowledge of X , so a coarse measurement of X tells you less and less about Y over time. You pay a fortune in present information for a forecast that only gets worse.

Now suppose an optimizer is acting on the system, driving it toward a target set T . Two things change at once, and both are in your favour.

First, *you no longer need the initial condition*. The optimizer drives a broad range of initial states all to T , so your uncertainty about X never propagates into uncertainty about Y . The $H(X)$ bits you would otherwise have had to acquire are simply irrelevant to the forecast.

Second, *naming the target is the forecast*. The laborious way to predict where the system ends up is to pin down its exact starting configuration and grind the dynamics forward step by step. For anything large that is hopeless: the starting configuration is enormous to write down, and chaos means the slightest error in it blows up until the forecast is worthless. The optimizer hands you a shortcut. Since it drives the system to the target wherever things began, you can throw away the starting configuration and just say where the system ends up: at the target. A target is a small thing to know, and a sturdy one, because the optimizer reaches it despite disturbances you could never have tracked.

This is a saving of effort, not a free lunch. Saying “it ends up at the target” *is* the forecast; you have not conjured extra knowledge out of nowhere. The point is only that knowing where a process is headed is cheap and stable, whereas reconstructing where it started, the sole alternative route to the same forecast, is vast and fragile. Optimizers are worth hunting for because they let you predict an outcome you would otherwise have almost no way to reach.

The courier makes the two routes concrete. To predict where the package ends up, you could try to track the courier’s vehicle, every traffic light, and every other car on the road, and integrate it all forward: a hopeless tangle of contingencies, chaotic besides, since one missed light reshuffles every turn after it. Or you could learn one fact, that the courier is delivering to 14 Elm Street, and predict that the package ends at 14 Elm Street, having modelled none of the traffic. The second route spends almost nothing and predicts almost perfectly.

Robustness sharpens the conclusion. Because the optimizer reaches T in spite of perturbations, your forecast “the system ends in T ” survives disturbances you could not have foreseen or measured. So you are spared not just the cost of the initial condition but the cost of modelling the perturbations too; the optimizer absorbs both. For this reason alone, an agent that cares only about prediction should scan its environment for optimizers and track what they steer toward, because nothing else packs the future into so short a description.

3.5 Optimization is an objective feature, not observer-relative

The argument of Section 3.4 answers a standing objection to treating “optimization” or “intelligence” as a real feature of the world at all.

The objection, stated at its strongest. A well-known view holds that intelligence, and optimization with it, is *observer-relative*: a system counts as intelligent only because it does well on the tasks *we* happen to care about, by the standards *we* happen to hold, and on a different choice of tasks the same system would look unremarkable. The view has a formal backbone in the No Free Lunch theorems, which show that, averaged over all possible problems, no optimizer beats blind search; competence is then never absolute, always relative to some restricted class of problems, and the choice of class reflects the observer’s interests. It has a philosophical sibling in Dennett’s intentional stance, on which calling a system an agent with goals is not a discovery about the system but a *stance* an observer takes up because it pays off predictively, so that whether something “has goals” depends on the observer’s modelling powers and convenience. Either way, “ X is an optimizer” looks less like a fact about X than like a relation between X and some particular onlooker.

Why the thermodynamic picture answers it. Notice first where we agree with the intentional stance: we too ground the significance of optimizers in their predictive value (Section 3.4). The disagreement is over whether that value is relative to a particular observer, and it is not. “Is something here driving this system into a narrow target, from a broad set of initial conditions and robustly to perturbation?” is a question about the system and the things acting on it, and its answer never mentions anyone’s preferences. The predictive leverage a known optimizer hands you, a short and accurate description of where the system is headed, is the same for every predictor no matter what, if anything, it wants. So you do not have to fix any tasks or standards to pick out optimizers: the bare aim of prediction, which any embedded agent has reason to pursue whatever its goals, already picks them out. The reason to care is observer-*independent*.

None of this contradicts No Free Lunch, which is about average competence across *all* problems and says nothing about whether some physical thing is driving a given system into a narrow target. Nor does it deny that an optimizer has a target: which target is itself a property of the system (where its dynamics actually funnel), not a verdict imposed from outside. The observer-relativity the objection worries about, whose standards get to judge success, never enters, because we are not grading the optimizer against our standards at all; we are noting that it has an attractor and using that fact, as any predictor could, to predict the world. Optimization is something the world does, not something an observer paints onto it.

To collect the conceptual frame before we turn to physics: an optimizer is a physical thing that drives the system it acts on into a convergent attractor (Section 3.2); the amount of optimization is the entropy it removes from that system (Section 3.3); and any predictor, whatever its goals, has an objective reason to track optimizers (Sections 3.4–3.5). What is left is to ask what physics permits. Optimization is entropy reduction in a subsystem, that is, *local* entropy reduction, and the next section shows that *global* entropy reduction is physically impossible. That tension, and the bookkeeping that resolves it, organizes the rest of these notes.

4 Reversibility and the second law

4.1 Microscopic physics is reversible

At the most fundamental level we can describe, the world does not forget. Take classical mechanics. The complete microscopic state of a system, its *microstate*, is the list of every particle’s position and momentum: a single point in the system’s *phase space*. The laws of motion carry each microstate along a unique trajectory, fixing the future from the present and, equally, the past from the present. So two distinct microstates can never flow into one: if they did, running the laws backward from the shared future state could not tell which to return to, and the laws permit no such ambiguity. Over any fixed interval, then, the evolution of an isolated system is a *bijection* on microstates, a perfect invertible pairing of starts with ends. Liouville’s theorem adds more: the evolution preserves not just distinctness but phase-space *volume*, so a blob of microstates can be stretched and folded without limit and yet keep its total volume. Quantum mechanics says the same in other notation: isolated evolution is unitary, hence invertible and volume-preserving. Either way, microscopic information is never created and never destroyed.

4.2 Why the funneling picture cannot hold globally

Now hold the convergent-attractor picture of optimization up against this reversibility. Taken literally at the microscopic level, the two flatly conflict. Funneling means many initial states end up in the same few target states: a many-to-one map. Reversibility says the microscopic dynamics are one-to-one. A perfect funnel is microscopically impossible.

There is a second, closely related obstruction: the second law of thermodynamics itself, that the entropy of an isolated system tends not to decrease. In the framework we are about to build, the second law is not an extra postulate but a *consequence* of reversibility, and seeing why illuminates both. The way out of the apparent paradox (optimization plainly happens, yet physics forbids funneling) is the central bookkeeping principle of these notes:

No physical process can reduce the entropy of an isolated system. Optimization is entropy reduction in a coarse-grained description of a subsystem, and the information squeezed out of the

optimized subsystem must be accounted for somewhere else in the larger system.

The remainder of this section makes the first sentence precise and proves it. Section 5 and Appendix B take up the second sentence: where the squeezed-out information can go, and how much squeezing an agent can do.

4.3 Coarse-graining: where probability enters a deterministic world

If the microscopic dynamics are deterministic and information-preserving, where do probability and rising entropy come from? From the fact that we never track the microstate. A gram of matter has on the order of 10^{23} coordinates, each to unbounded precision; no observer, and no embedded agent, can hold that. What we track instead is a *coarse-grained* description. Partition the phase space into discrete cells, each lumping together all the microstates we cannot tell apart (every particle’s position and momentum to finitely many digits, say, or just a few macroscopic variables), and describe the system by naming the cell it occupies.

The dynamics of cells, unlike the dynamics of microstates, are genuinely stochastic. The current cell does not fix the next one, because different microstates inside it flow to different places. The best we can write down is a transition probability $P(y, x)$: the fraction of cell x , measured by phase-space volume, that lands in cell y one step later. This randomness is not metaphysical; it is the shadow of the microscopic detail we declined to resolve. Chaotic dynamics keep pulling that detail up across the resolution scale, so the coarse trajectory looks like a string of random jumps even though the flow beneath it is deterministic.

One structural assumption underlies all of stochastic thermodynamics: the coarse trajectory is a *Markov process*, meaning the distribution of the next cell depends only on the current cell and not on the earlier history. Intuitively, the coarse-graining is good enough that the current cell holds everything about the past that matters for the coarse future, and the discarded detail acts like fresh noise at each step rather than hidden memory. Whether a given coarse-graining is really Markovian is a deep and still-open question, but two reassurances help. First, there are exactly solvable models, the *multibaker maps*: deterministic, time-reversible, chaotic systems whose coarse-grainings provably reproduce arbitrary Markov chains, with all the randomness sitting in the initial condition. They show that macroscopic stochasticity and irreversibility are fully compatible with microscopic determinism and reversibility. Second, the Markov property is what makes everyday statistics work, and its *failure backward in time* is the arrow of time. A dropped glass shatters at a predictable moment, and its shards follow well-defined local statistics that owe nothing to events at the neighbor’s house. Run the film backward and that breaks: to retrodict when a shattered vase was dropped, local statistics are useless, and your best clue might be a conversation about the accident taking place next door. Forward evolution obeys memoryless local laws; backward evolution does not. That asymmetry is exactly what the Markov property encodes.

One more consequence of reversibility, and then the second law. Liouville’s theorem makes phase-space volume a *stationary measure* of the chain: weight each cell by its volume, and one step carries that weighting to itself. We take all cells to have equal volume (a *mesoscopic* coarse-graining, what you get by truncating every coordinate to fixed precision). Stationarity of the uniform weighting then pins down the transition probabilities. With π constant, the condition $\sum_x P(y, x) \pi(x) = \pi(y)$ cancels to $\sum_x P(y, x) = 1$ for every y : the probabilities flowing *into* each cell sum to one, just as those flowing *out of* each cell do. A matrix with both properties is *doubly stochastic*. This is the fingerprint reversibility leaves on the coarse dynamics: a doubly stochastic step can shuffle and spread probability but never concentrate it, since concentration would pack several cells’ worth of volume into fewer cells, which Liouville forbids.¹

4.4 The second law of thermodynamics

Theorem 4.1 (Second law for doubly stochastic chains). *Let P be a doubly stochastic transition matrix on a countable state space, let the random variable X (the coarse-grained state now) have distribution μ , and let Y (the state one step later) have distribution $P\mu$, where $(P\mu)(y) := \sum_x P(y, x) \mu(x)$. Then*

$$H(Y) \geq H(X).$$

¹Everything in these notes generalizes to coarse-grainings with unequal cell volumes: one measures entropy *relative to* the stationary volume measure π , replacing $H(\mu)$ by $H_\pi(\mu) := \sum_x \mu(x) \log \frac{\pi(x)}{\mu(x)}$ and the codelength by $\hat{H}_\pi(x, \mu) := \log \frac{\pi(x)}{\mu(x)}$. This is standard, but not needed for anything here; equal cells keep the notation light at no conceptual cost.

Proof. We prove the statement in two steps.

Step 1: one step of any Markov chain brings distributions closer together. Let μ and ν be any two distributions (or nonnegative measures) on the state space. We claim the data processing inequality

$$D(P\mu \| P\nu) \leq D(\mu \| \nu).$$

To see this, fix an output state y and apply the log-sum inequality (Lemma 2.9) to the numbers $a_x = P(y, x)\mu(x)$ and $b_x = P(y, x)\nu(x)$, whose sums over x are $P\mu(y)$ and $P\nu(y)$ respectively:

$$\sum_x P(y, x)\mu(x) \log \frac{P(y, x)\mu(x)}{P(y, x)\nu(x)} \geq P\mu(y) \log \frac{P\mu(y)}{P\nu(y)}.$$

Now sum this inequality over all y . On the left-hand side, the factor $\log \frac{\mu(x)}{\nu(x)}$ does not depend on y , and $\sum_y P(y, x) = 1$ because P is stochastic, so the left-hand side totals $\sum_x \mu(x) \log \frac{\mu(x)}{\nu(x)} = D(\mu \| \nu)$. The right-hand side totals $D(P\mu \| P\nu)$. This proves the claim. The intuition is exactly the coding one: passing two information sources through the same noisy channel can only make them harder to tell apart.

Step 2: choose the reference measure to be uniform. Let $\sharp$ denote the counting measure, $\sharp(x) = 1$ for every x (an unnormalized uniform measure; Lemma 2.9 never required normalization). Double stochasticity of P says precisely that $P\sharp = \sharp$, since $(P\sharp)(y) = \sum_x P(y, x) = 1$. Moreover, divergence from the counting measure is just negative entropy:

$$D(\mu \| \sharp) = \sum_x \mu(x) \log \frac{\mu(x)}{1} = -H(\mu).$$

Applying Step 1 with $\nu = \sharp$:

$$-H(P\mu) = D(P\mu \| P\sharp) \leq D(\mu \| \sharp) = -H(\mu),$$

which rearranges to $H(P\mu) \geq H(\mu)$. □

Notice what the proof used: the list is short, and each item is physical. We used the Markov property, to write one step as a transition matrix, and double stochasticity, which came from Liouville’s theorem, that is, from the *reversibility* of the microscopic dynamics. Nothing else. So macroscopic irreversibility, the rise of entropy, is not in tension with microscopic reversibility; it follows from it, once we grant that observers track coarse cells rather than microstates. That is the precise content of the slogan “the second law follows from the reversibility of physics”.

4.5 What the second law does and does not forbid

Three clarifications will prevent misreadings that matter later.

First, Theorem 4.1 applies to an *isolated* system evolving on its own: the entropy of the whole cannot fall. It says nothing against the entropy of a *subsystem* falling, and the entropy of subsystems falls all the time; that is what refrigerators, crystallization, house-builders, and every other optimizer accomplish. What the theorem imposes is a bookkeeping constraint on the whole: when a subsystem’s entropy drops, the books must balance, with at least as much entropy appearing elsewhere in the total system. The complete classification of where “elsewhere” can be is the subject of Section 5.

Second, the theorem as stated concerns the entropy of the *ensemble*, the probability distribution μ , and it holds on average and in distribution; individual trajectories can pass through improbable, low-codelength states. A sharper, trajectory-level version of the second law, with explicit and very small bounds on the allowed fluctuations, becomes available in the algorithmic framework of Section 7.

Third, and this is what the rest of the notes turn on: the theorem assumed a fixed dynamics P uninfluenced by any observer, and an entropy defined relative to a given distribution μ . Agents complicate both assumptions. An agent that *observes* the system and conditions its behavior on what it sees is not a fixed P ; how much that buys, and what it costs, is the subject of Appendix B. And the apparently innocent reliance on a given μ conceals a deep conceptual problem, the subject of Section 6. But first we take up the bookkeeping question directly: when an agent lowers a subsystem’s entropy, where can the displaced entropy go? The next section gives the complete answer. (A self-contained toy model in which every step can be checked by hand is developed separately in Appendix A.)

5 Three types of optimization under information conservation

5.1 The bookkeeping problem

Now for the global accounting. Decompose the universe into three parts: the agent A , the subsystem S that the agent is trying to optimize, and the rest of the environment E . Optimization of S means reducing $H(S)$: funneling the subsystem from many possible configurations toward a narrow target set. The second law (Theorem 4.1, applied to the isolated whole) says the joint entropy of (A, S, E) cannot decrease, and the reversibility that underlies it says information about S 's initial condition cannot simply vanish. So the bookkeeping question is: in what ways can an agent reduce $H(S)$ while the global books remain balanced? Daniel C and Ebtekar's analysis identifies exactly three channels, and the classification is exhaustive: entropy squeezed out of S must end up in E , end up in A , or have been pre-paid.

5.2 Type 1: dump waste heat into the environment

The first channel is the everyday one: transfer the subsystem's entropy into the environment. The agent arranges an interaction between S and E under which $H(S)$ falls while $H(E)$ rises by at least as much. A refrigerator narrows the distribution of its interior's thermal state while heating the kitchen behind it. A house-building team imposes order on lumber and nails while dissipating heat, exhaust, and scattered debris into the surroundings. In the coin world of Appendix A, this is a conditional-swap circuit that moves uncertainty from designated coins into other coins. The signature of Type 1 is that the environment absorbs the entropy: the agent is a broker arranging the transfer, and the agent's own state need not change appreciably.

5.3 Type 2: absorb the entropy into the agent's memory (measurement)

The second channel is subtler: the agent reduces $H(S)$ by *measuring* S and storing the outcomes, so that the subsystem's entropy is transferred into the agent's own memory. This is the operating principle of one of the most famous thought experiments in thermodynamics.

Maxwell's demon. A room full of gas is divided in two by a partition, and the partition has a small door that can be opened and closed at will. A tiny intelligence (the "demon") watches the molecules. Whenever a molecule approaches the door from the left, the demon opens it; whenever one approaches from the right, the demon closes it. Molecule by molecule, the gas accumulates on the right side of the room. The entropy of the gas has plainly decreased (we are far less uncertain about the molecules' whereabouts than before), no work appears to have been done, and no heat has been dumped anywhere. Has the second law been violated?

It has not, and the reversibility argument of Section 4.1 shows exactly where the missing entropy went. Consider the end of the process and pick any particle now on the right side. Two distinct histories are consistent with its presence there: either it began on the right and stayed, or it began on the left and the demon admitted it. These two histories produce *identical* final gas configurations, so the gas alone cannot distinguish them. But the microscopic dynamics of the whole system are reversible, and reversibility demands that the past be reconstructible from the present. The only way the total system can remain reversible is if something else differs between the two histories, and that something is the demon's memory: the record of its observations and door operations. For every bit of entropy the demon removes from the gas, at least one bit of entropy accumulates in the demon's memory. The gas's order is purchased with the disorder of the demon's records; globally, nothing has decreased. (We will redo this argument with full rigor, for individual states rather than ensembles, in Section 7.4.)

The signature of Type 2 is that the agent absorbs the entropy: measurement moves uncertainty from the world into the measurer. The demon's memory is finite, so this channel is exhaustible. When the memory fills, the demon must either stop or clear it, and clearing it ultimately exports the stored entropy to the environment as Type-1 waste after all (the exact bookkeeping is given in Section 7.4). Type 2 is a buffer, not a bottomless sink.

5.4 Type 3: spend mutual information you already have

The first two channels balance the books by increasing entropy somewhere (in E , or in A). The third channel is the surprising one: it reduces $H(S)$ *without increasing entropy anywhere*, by consuming a correlation that already exists between the agent and the subsystem.

Suppose that at time t , the agent’s state and the subsystem’s state share mutual information $I(S_t; A_t) > 0$: the agent already “knows something” about the subsystem, in the purely statistical sense of Definition 2.6. For clean accounting, assume the environment is independent of both, so that the joint entropy decomposes (by the definition of mutual information and the chain rule) as

$$H(S_t, A_t, E_t) = H(A_t) + H(S_t) - I(S_t; A_t) + H(E_t).$$

Now the agent performs an operation with the following effects: it *erases the copy of the shared information that resides inside S* , using its own copy as the key, while leaving its own marginal state and the environment untouched. The operation transforms the subsystem so that

$$H(S_{t+1}) = H(S_t) - I(S_t; A_t), \quad I(S_{t+1}; A_{t+1}) = 0, \quad H(A_{t+1}) = H(A_t), \quad H(E_{t+1}) = H(E_t).$$

Adding up the joint entropy after the operation:

$$H(A_{t+1}) + H(S_{t+1}) + H(E_{t+1}) = H(A_t) + H(S_t) - I(S_t; A_t) + H(E_t),$$

which is *exactly* the joint entropy from before. The subsystem’s entropy has genuinely fallen by $I(S_t; A_t)$ bits, total entropy is unchanged, no heat was produced, and no memory filled up. What was consumed is the correlation itself: afterward, $I(S_{t+1}; A_{t+1}) = 0$, and the trick cannot be repeated without first re-acquiring mutual information.

Example 5.1 (Type 3 in one bit: the controlled-NOT). The smallest possible instance makes the book-keeping transparent. Let the subsystem hold a single uniformly random bit s , and let the agent hold a perfect copy, $a = s$. Then $H(S) = 1$, $H(A) = 1$, $I(S; A) = 1$, and the joint entropy is $H(S, A) = 1$ bit (two equally likely joint states, 00 and 11). The agent now applies a *controlled-NOT*: flip s if $a = 1$, do nothing if $a = 0$. In both joint states the result sets s to 0: the subsystem becomes deterministic, $H(S_{t+1}) = 0$, a full bit of entropy reduction. The operation is reversible (apply it again to undo it), touches no environment, and produces no heat. And the joint entropy is still exactly 1 bit, now carried entirely by the agent’s own marginal (a remains a uniform bit, uncorrelated with the now-deterministic s). The correlation was spent: a second application of the trick is useless, since $I(S_{t+1}; A_{t+1}) = 0$.

5.5 The demon reframed, and the recap

The Type 3 channel recasts Maxwell’s demon in an illuminating way. The demon’s measure-and-control behavior can be decomposed into two distinct steps: a *copy* step, in which the demon interacts with the gas to acquire mutual information with it (this is Type 2: the demon’s memory absorbs entropy, or more precisely, becomes correlated with the gas), followed by a *control* step, in which the demon uses that mutual information to steer the gas into a narrower distribution (this is Type 3: the correlation is spent, and the control step itself produces no further entropy anywhere). Measurement is the purchase of correlation; steering is its expenditure. How much entropy reduction can the control step buy per bit of correlation acquired in the copy step? That is exactly the question answered by the *Touchette–Lloyd theorem* (Appendix B): it bounds the steering achievable in a Type-3 step by the mutual information available to spend, bit for bit. (Section 7.4 will later derive an exact algorithmic version of the same bound.)

To summarize, an embedded agent can optimize a subsystem in exactly three ways:

1. **Type 1:** dump the subsystem’s entropy into the environment as waste heat;
2. **Type 2:** absorb the subsystem’s entropy into its own memory through measurement;
3. **Type 3:** spend mutual information it already shares with the subsystem, erasing the subsystem’s copy of the correlation.

The three channels compose: a realistic optimizer cycles among them, measuring (Type 2), steering (Type 3), and periodically flushing its memory to the environment (Type 1) to make room for the next round.

Type 3 raises a quantitative question, answered in Appendix B: if steering is paid for by spending mutual information shared with the subsystem, exactly how many bits of entropy reduction does each bit of mutual information buy? The Touchette–Lloyd theorem, developed there, gives the bound (one bit of entropy reduction beyond the blind baseline per bit of mutual information between action and environment), and we draw on its statement in what follows.

6 The trouble with subjective entropy

Everything so far has treated the distribution μ , the thing all our entropies are defined against, as simply given. This section pokes at that assumption, and finds that it breaks down right where agent foundations needs it to hold. The critique, and its resolution in the next section, follow Daniel C and Ebtakar’s essay and the foundational paper of Ebtakar and Hutter.

6.1 Entropy is supposed to measure the capacity to optimize

Start with what entropy is *for*. Across physics and engineering, entropy’s job is to measure a system’s capacity to do work. Everything so far (Sections 4–5, and the coin world of Appendix A) lets us read “capacity to do work” as *capacity to optimize*: negentropy is the fuel that funnels broad distributions into narrow ones.

Apply this to the demon after a long run of Type-2 optimization. Its memory holds, say, m bits, some now filled with measurement records. The demon’s *remaining* capacity to optimize the gas is its free memory: the total size minus the entropy already stored. Each further bit of optimization eats another bit of free memory. Here is the question everything turns on: *is the demon’s free memory an objective, physical quantity?* It surely ought to be. Whether the hardware can absorb another billion measurements is a fact about the demon’s physical configuration, with engineering consequences (how much longer it can refrigerate, how much heat it must eventually pay to clear the records). It would be strange for the demon’s actual ability to act on the gas to depend on the beliefs of some outside observer describing it. Yet that is exactly what the standard formalism delivers.

6.2 The exogenous distribution

Recall the shape of the standard definitions (Definition 2.4). The entropy of an *individual* state x is undefined. What is defined is the Shannon codelength $\hat{H}(x, \mu) = \log \frac{1}{\mu(x)}$ of a state *relative to a distribution* μ , together with its average, the Gibbs–Shannon entropy $H(\mu)$. The distribution μ is an input, not an output: it encodes somebody’s knowledge of the system, and the formalism never says whose knowledge, where it is stored, how it was obtained, or what it cost. The standard framework, in our terms, treats knowledge as *exogenous*.

At the great equilibrium successes of thermodynamics this exogeneity is harmless, because the right μ is essentially forced. But “harmless” rests on three conditions. Call μ a *useful summary* of an actual physical state x when all three hold:

1. **Simplicity:** μ has a short description, such as “an ideal gas of N molecules at temperature 300 K and pressure 1 atm, at equilibrium”. (Otherwise, specifying μ itself smuggles in unaccounted-for information; hold this thought.)
2. **Concentration:** the codelength concentrates near its mean, $\hat{H}(X, \mu) \approx H(\mu)$ with high probability under $X \sim \mu$. For systems composed of many weakly interacting parts, this follows from the law of large numbers; it is what lets a single number, “the entropy”, stand in for a whole distribution of codelengths.
3. **Typicality:** the actual state x is an unremarkable sample from μ , falling in the high-probability set where the previous two conditions bite.

When all three conditions hold, $H(\mu) \approx \hat{H}(x, \mu)$, every reasonable notion of entropy agrees, and one may do thermodynamics with macrovariables alone, never asking where μ lives. Equilibrium statistical mechanics

resides entirely inside this regime, and the classical apparatus of Clausius and Boltzmann (temperature, pressure, chemical composition as the privileged macrovariables) is well adapted to it.

6.3 Three ways the ensemble picture fails

Far from equilibrium, and above all for *information-bearing* states, each of the three conditions can fail on its own, and each failure breaks the formalism in its own instructive way.

Failure of simplicity: knowledge with no ledger. Let μ be a point mass on one particular, intricate configuration x (“I know the exact microstate”). Then $H(\mu) = 0$, and the formalism declares the system fully known, hence fully available for work extraction: knowing the exact state of a gas, one can in principle extract all of its energy as work, using a transformation tailored to that state. But the tailored transformation, and the knowledge it embodies, had to be specified by *something*; the description of μ is exactly as intricate as x itself, and the formalism books it at zero cost. The Gibbs–Shannon account permits arbitrarily detailed knowledge as a free exogenous resource. Physically, any agent in possession of such knowledge must encode it in a memory, and acquiring, storing, and clearing that memory have costs that the second law cares about, as the demon already showed us. The ensemble formalism has no slot for these costs.

Failure of concentration: averages that describe no state. A robot flips a hidden fair coin and, depending on the outcome, either fully drains its battery or leaves it fully charged. Under the resulting fifty-fifty mixture μ , the Gibbs–Shannon entropy of the battery-plus-surroundings sits exactly halfway between the entropy of the charged state and that of the drained state, and a free-energy calculation based on $H(\mu)$ concludes that the robot can perform about half a charge of work. This is false in every possible world: the robot can perform either a full charge of work or none at all. The mean $H(\mu)$ is an average over an ensemble whose members behave nothing like the average, because the distribution is bimodal rather than concentrated. The number we actually want is a property of the *individual* state the robot is in fact in, and the ensemble formalism does not define such a number.

Failure of typicality: structure the ensemble cannot see. Suppose the actual state x , while formally a possible sample from the equilibrium ensemble μ , happens to be special in a way the macrovariables do not capture: the gas particles, say, momentarily occupy a strongly patterned, compressible configuration. The codelength $\hat{H}(x, \mu)$ prices x as if it were generic, and a work calculation based on it will *underestimate* what a sufficiently clever machine (one that recognizes and exploits the pattern, that is, a machine running a compression algorithm) can extract from x . Taken at face value, the ensemble calculation would brand such a machine a violator of the second law. The correct response is not to outlaw clever machines but to admit that the entropy relevant to what can be done with x is a property of x ’s actual structure, not of the ensemble we happened to embed it in.

6.4 The demon’s capacity, and why this is the central case for agent foundations

Return now to the demon’s free memory, and the puzzle lands precisely. The demon’s remaining optimization capacity is m minus the entropy of its memory contents, but the entropy of its memory contents *relative to which distribution*? If we, the observers, refine our beliefs about what the demon has recorded, the standard formalism says the entropy of the demon’s memory drops, and the demon’s “capacity to do work” rises, though nothing about the demon’s hardware has changed. The classical macrovariable repertoire is no help either: temperature, pressure, and chemical composition are designed for systems whose unresolved degrees of freedom mix rapidly and forgettably, whereas a memory is, by engineering design, a system whose states are stable, distinguishable, and meaningful. Every distinct memory configuration must be treated as its own condition; there is no thermalizing ensemble over memory states for μ to latch onto, and there is no privileged ensemble over outputs of a long computation, because the whole point of computation is to produce states whose structure no simple prior anticipates.

For agent foundations, these are not pathological corner cases to be quarantined. They are the entire subject matter. The states we care about (an agent’s memory and world model, the records left by measurements, computations in progress, an environment halfway through being reshaped toward a target) are

exactly the far-from-equilibrium, information-bearing, intricately structured states on which the ensemble formalism fails. If entropy is to serve as the measure of optimization capacity for embedded agents, we need a definition of entropy that applies to an *individual physical state*, with no exogenous distribution anywhere in sight. Remarkably, such a definition exists, and it comes from the theory of computation.

7 Algorithmic thermodynamics

Algorithmic thermodynamics, developed by Ebtekar and Hutter on foundations from Bennett, Zurek, Gács, and Levin, keeps the entire coarse-grained Markov framework of Section 4 and changes one thing: it replaces the subjective ensemble μ with universal computation. Instead of asking “how surprising is x under μ ?”, it asks “how hard is x to describe, full stop?” The payoff is conceptual: an objective, observer-free entropy that *dissolves the subjectivity problem* of Section 6, defined for a single physical state with no ensemble in sight. Along the way we get a second law and fluctuation bounds for individual trajectories, and an exact account of Maxwell’s demon.

7.1 Kolmogorov complexity

Fix a universal computer U ; concretely, think of an interpreter for your favorite general-purpose programming language. Every coarse-grained state x of a physical system is identified, via the coarse-graining’s encoding, with a finite binary string, so it makes sense to talk about programs that output x .

Definition 7.1 (Description complexity). The *Kolmogorov complexity* (or description complexity) $K(x)$ of a string x is the length, in bits, of the shortest program that outputs x when run on U . The *conditional complexity* $K(x | y)$ is the length of the shortest program that outputs x when given y as auxiliary input. The *algorithmic mutual information* between two strings is

$$I(x : y) := K(x) + K(y) - K(x, y),$$

the number of bits that a description of one saves in describing the other.²

The right mental model for $K(x)$ is *optimal lossless compression*: the size of the smallest self-contained representation from which x can be regenerated exactly. A few examples fix the scale. A million zeros has tiny complexity: a fifteen-byte program prints it. The first million digits of π have tiny complexity too, though they pass every statistical test for randomness, because a short program computes them. A million bits straight from a quantum random-number generator have complexity close to a million bits: almost surely there is no structure to exploit, and the shortest “program” is essentially the string quoted verbatim. A state of 10^{23} particles all in one corner of a box has low complexity (a short loop prints the repeated coordinates); a generic thermalized state has complexity near the full length of its encoding. The slogan: *low entropy is compressibility*, a property of the individual state x , with no ensemble in sight.

Algorithmic mutual information behaves the way Section 2’s mutual information does, but for individual objects: $I(x : x) \stackrel{+}{=} K(x)$ (a copy of x saves you everything), $I(x : y) \stackrel{+}{=} 0$ for typical independent random strings (a description of one is useless for the other), and intermediate values measure partial correlation, such as between a measurement record and the system it measured.

Because the shortest program may be arbitrarily hard to find, statements about K hold up to additive constants reflecting fixed wrapper code; following Ebtekar and Hutter we write $f \stackrel{+}{<} g$ for $f \leq g + c$, $f \stackrel{+}{>} g$ for $f \geq g - c$, and $f \stackrel{+}{=} g$ for both, where c depends only on fixed choices like the universal computer, never on the strings involved.

²Two pieces of technical fine print, recorded once and then suppressed. First, programs are required to be *self-delimiting*: no valid program is a prefix of another, so a program announces its own end. This makes program lengths behave like codeword lengths; in particular they satisfy the Kraft inequality $\sum_x 2^{-K(x)} \leq 1$, which is what lets $2^{-K(x)}$ act like a (sub)probability distribution. Second, the chain rule for K holds in the form $K(x, y) \stackrel{+}{=} K(y) + K(x | y, K(y))$, with the complexity of y itself appearing in the conditioning; consequently some identities below, including the second expression for $I(x : y)$, implicitly carry such terms. Every statement we make is correct with these refinements installed, and none of them matter at physical scales.

7.2 Why K deserves to be called entropy

Three families of facts justify treating $K(x)$ as *the* entropy of the individual state x .

The choice of computer is physically negligible. K depends on the universal computer U , which looks alarming for a proposed physical quantity. But the dependence is bounded by translation: for any two universal computers, a fixed interpreter converts programs for one into programs for the other, so the two versions of K differ by at most the interpreter’s length, uniformly in x . To gauge the scale, convert to physical units: entropy in bits converts to thermodynamic entropy at $1 \text{ bit} = k_B \ln 2 \approx 9.57 \times 10^{-24} \text{ joules per kelvin}$. Suppose two programming languages were so alien to each other that translating between them required a 12-gigabyte interpreter, far larger than any real interpreter. Twelve gigabytes is about 10^{11} bits, so the two languages would assign every physical system entropies agreeing to within 10^{-12} J/K , far below experimental resolution for macroscopic systems. For all physical purposes, K is computer-independent.

K agrees with the Shannon picture exactly where the Shannon picture is valid. Let μ be any computable probability distribution. Then for *every* state x ,

$$K(x \mid \mu) \stackrel{+}{\leq} \log \frac{1}{\mu(x)} = \hat{H}(x, \mu),$$

because one way to describe x , given access to μ , is to transmit its Shannon codeword (Section 2.3); the shortest description can only be shorter. In the reverse direction, a counting argument based on the Kraft inequality shows that the inequality is nearly tight for nearly all samples: for any $\delta > 0$, with probability at least $1 - \delta$ over $X \sim \mu$,

$$K(X \mid \mu) \stackrel{+}{\geq} \log \frac{1}{\mu(X)} - \log \frac{1}{\delta}.$$

In words: the optimal universal description of a typical sample is its Shannon codeword, up to logarithmically small slack; atypical samples can beat the Shannon codeword, but only a δ -fraction can beat it by more than $\log \frac{1}{\delta}$ bits. Averaging the two displays recovers Zurek’s identity

$$H(\mu) \stackrel{+}{=} \langle K(X \mid \mu) \rangle_{X \sim \mu} :$$

the Gibbs–Shannon entropy is the mean Kolmogorov complexity of a sample, given prior knowledge of the distribution. Two specializations sharpen this. Taking μ uniform on a finite set B shows that for any simply describable set B containing x , we have $K(x) \stackrel{+}{\leq} K(B) + \log |B|$, with near-equality for the vast majority of elements; taking B to be a Boltzmann macrostate (the set of all microstates sharing given macrovariable values) recovers the Boltzmann entropy $\log |B|$ as an upper bound on K , achieved by typical members. And the bound applies to *any* simple set, not just classical macrostates: Ebtekar and Hutter’s example is that the entropy of a bookshelf can be estimated by taking B to be the set of configurations compatible with how the books are sorted. The algorithmic entropy automatically considers every simple description of the state, classical macrovariables included, and charges the state the cheapest one.

These facts settle the three failure modes of Section 6.3. Where the ensemble picture is valid (simple μ , concentration, typical x), $K(x) \approx \hat{H}(x, \mu) \approx H(\mu)$, and algorithmic entropy reproduces the classical answers; nothing is lost. Where the ensemble picture fails, K keeps working: a point-mass μ on an intricate state no longer yields zero entropy, because $K(x)$ is large regardless of what distribution we mention alongside it, so the “free knowledge” loophole closes (the knowledge is now priced, in the only currency that matters, description length); the robot’s battery gets a definite per-state answer, $K(\text{charged state})$ or $K(\text{drained state})$, with the ensemble value revealed as a mere average of the two by Zurek’s identity; and a secretly patterned gas configuration gets credited for its structure, $K(x) \ll \hat{H}(x, \mu)$, so the clever compression machine extracts exactly the work that x ’s actual description length permits, violating nothing.

K is uncomputable, and a second law requires this. No algorithm computes $K(x)$ from x . One direction of approximation is available: by running ever more candidate programs for ever longer, one obtains a decreasing sequence of upper bounds converging to $K(x)$ (in the jargon, K is upper semicomputable), which

is why real-world compressors give genuine upper bounds on entropy. But no algorithm can certify large *lower* bounds on complexity: there is no procedure that, given x , verifiably reports “ $K(x)$ is at least one million”. (This is Chaitin’s incompleteness theorem, and the proof is one line of self-reference: if such a certifier existed, then a short program could enumerate strings until it finds one certified to have complexity at least a million and print it, thereby describing a supposedly million-bit-complex string with a few hundred bits, a contradiction.)

At first sight uncomputability looks like a defect in a proposed physical quantity. It is in fact load-bearing. Suppose we tried to do thermodynamics with any entropy-like state function K' for which an algorithm *could* certify large values. Then a short program p could search for and output a state x^* certified to satisfy $K'(x^*) \gg 0$. But any short program that can *construct* x^* can also *erase* it: run p to produce a second copy of x^* alongside the existing one, use the copy to reversibly cancel the original (a controlled-NOT for every bit, exactly as in Example 5.1), then run p backward to clean up. The net effect of a constant-size machine is to take the world from a state containing x^* (with K' -entropy $\gg 0$) to a state not containing it (K' -entropy ≈ 0). A small, simple, reversible machine that can destroy unbounded amounts of “entropy” at will is a perpetual motion machine with respect to K' : no second law can hold for any computable notion of entropy. The uncomputability of K is precisely what closes this loophole; it guarantees that no simple machine can systematically recognize which states are simple-but-disguised, and therefore none can systematically un-disguise them. (Note the parallel with Appendix B.8: knowledge that cannot be obtained cannot be spent.)

7.3 The algorithmic second law

Return to the physical setting of Section 4: a coarse-grained state space with equal-volume cells, evolving as a Markov chain with doubly stochastic, *computable* transition probabilities (the laws of physics are assumed to have a short program, a complexity-theoretic version of the Church–Turing thesis; we choose our reference computer so that the dynamics are simply describable). In this setting the algorithmic entropy of the system in state x is just $K(x)$, and it obeys a second law, which descends from Levin’s law of *randomness conservation*.

Theorem 7.2 (Algorithmic second law; Levin, Ebtekar–Hutter, stated informally). *Let (X_t) be a computable, doubly stochastic Markov chain modeling an isolated system’s coarse-grained evolution, and let $s < t$ be two times. Then for every $\delta > 0$, with probability greater than $1 - \delta$,*

$$K(X_s) - K(X_t) \stackrel{+}{<} K(t - s) + \log \frac{1}{\delta}.$$

That is: *the description complexity of an isolated system’s state almost never decreases, except by a precisely priced fluctuation allowance*. The theorem improves on the ensemble second law (Theorem 4.1) in two ways that matter for us. It holds for each individual trajectory with high probability, not merely for the average of an ensemble; and it requires no initial distribution μ at all, removing the exogenous ensemble exactly as Section 6 required.

The two terms of the allowance are each interpretable, and each is very small in physical terms.

The term $\log \frac{1}{\delta}$ prices *chance* fluctuations: rare trajectories on which entropy temporarily falls. Its scale is set by how rare a fluctuation you care to consider. Tolerating events of probability $\delta = 2^{-1000}$, the allowed entropy dip is about a thousand bits, which in physical units (recall 1 bit $\approx 10^{-23}$ J/K) is around 10^{-20} J/K: twenty orders of magnitude below anything measurable. The statistical character of the second law is real, but quantitatively trivial at macroscopic scales.

The term $K(t - s)$, the complexity of the *elapsed time*, prices *deterministic recurrences*, and its necessity is subtle. The Poincaré recurrence theorem guarantees that an isolated finite system eventually returns arbitrarily close to its initial low-entropy state, so entropy cannot literally always increase. How does the theorem survive? Because recurrence times are astronomically long and arithmetically structureless: a system whose recurrence happens at step $t - s = 17,382, \dots$ (imagine 10^{20} digits) revisits low entropy only at moments whose timestamps are themselves about as complex as the state, and the $K(t - s)$ allowance covers exactly this. For any time interval you can *simply specify* (10^9 years, 2^{100} Planck times), $K(t - s)$ is a few hundred bits at most, and entropy, regarded as a function of simply specified times, increases monotonically up to negligible slack.

One special case recurs throughout the demon analysis below.

Corollary 7.3 (Deterministic dynamics cannot change entropy). *If the evolution over the interval is a computable bijection f with a short description (for instance, a few steps of simply describable reversible dynamics), then*

$$K(f(x)) \stackrel{\pm}{=} K(x).$$

The reason is immediate from the compression picture: given the dynamics, a description of x is a description of $f(x)$ and vice versa, so the two complexities differ by at most the (small) complexity of f itself. The corollary has an important consequence: *substantial entropy production requires randomness*. A deterministic, simply describable evolution can shuffle states but cannot make them more complex by more than a constant; in classical physics, the randomness that drives entropy production is supplied by coarse-graining chaotic microdynamics (Section 4.3), which continually injects effectively fresh random bits into the coarse description. A useful way to picture it: a doubly stochastic evolution is equivalent to drawing a random bijection at each step (say, by tossing coins), and entropy can increase by at most the number of coins tossed, and only to the extent that the coins are independent of the state. The Shannon-level second law is blind to this distinction between deterministic reshuffling and genuine randomization; the algorithmic version sees it.

7.4 Maxwell’s demon, exactly

We now redo the demon, this time as theorem rather than story, and recover the Touchette–Lloyd bound as a statement about individual physical states. Let the demon’s memory and the gas be jointly coarse-grained, writing joint states as pairs (memory contents, gas state). The demon’s memory starts in a blank reference state 0; the gas starts in some state x .

Full measurement, then erasure. In the idealized case the demon performs a complete measurement, reversibly copying the gas’s state into memory:

$$(0, x) \mapsto (x, x),$$

and then, using its copy as the control of a conditional operation (a string of controlled-NOTs, Example 5.1), reversibly resets the gas to a reference state:

$$(x, x) \mapsto (x, 0).$$

Each map is injective on the states that can actually occur (the first on pairs with blank memory, the second on pairs whose two slots agree), hence extendable to a bijection of the joint state space, and each is simply describable. Corollary 7.3 therefore applies to both steps, and the *total* complexity never moves:

$$K(0, x) \stackrel{\pm}{=} K(x, x) \stackrel{\pm}{=} K(x, 0) \stackrel{\pm}{=} K(x).$$

Look at what these constants say. The gas has gone from complexity $K(x)$ to complexity $\stackrel{\pm}{=} 0$: a complete Type-2 optimization. The joint complexity never changed: the second law was never threatened, at any step, without any need to “complete a cycle” or invoke ad hoc accounting. And the balance is now carried entirely by the demon’s memory, which holds a record of complexity $K(x)$. The gas’s erasure was permitted *precisely because a copy existed*; formally, “clear slot two given that slot one holds a copy” is injective. The step that the second law forbids is clearing the *last* copy:

$$(x, 0) \mapsto (0, 0)$$

is either not injective (if it is to work for every x) or not simply describable (if hard-wired for one particular complex x , the wiring itself would have complexity $K(x)$, and Corollary 7.3 charges for the dynamics’ complexity). To get its memory back, the demon must clear that last copy, irreversibly destroying $K(x)$ bits of complexity; it cannot do this for free, and the only lawful route is to export those $K(x)$ bits into the environment as Type-1 waste (Section 5.2). The classic resolution of the demon paradox (that the demon’s own information processing is what rescues the second law) is here not an extra postulate but a direct consequence of the complexity bookkeeping.

Partial measurement: the algorithmic Touchette–Lloyd bound. Realistic demons measure only a little. Let the measurement be $m(x)$, where m is any simply describable (possibly random, possibly many-to-one) function of the gas state: a few bits about one approaching molecule, say. The demon records the measurement, then uses the record as a control to drive the gas from x to some new state y :

$$(0, x) \mapsto (m(x), x) \mapsto (m(x), y).$$

Whatever clever feedback the second step implements, as long as it is a simply describable mixture of reversible operations, the algorithmic second law applied to the *joint* system gives

$$K(m(x), x) \stackrel{+}{<} K(m(x), y).$$

Expand both sides with the chain rule ($K(m, \cdot) \stackrel{\pm}{=} K(m) + K(\cdot | m)$, fine print as in Definition 7.1), subtract $K(m(x))$ from both sides, and rewrite conditional complexities via mutual information ($K(x | m) \stackrel{\pm}{=} K(x) - I(m : x)$):

$$K(x) - I(m(x) : x) \stackrel{+}{<} K(y) - I(m(x) : y) \leq K(y),$$

where the last step uses nonnegativity of algorithmic mutual information. Rearranged:

$$\boxed{K(x) - K(y) \stackrel{+}{<} I(m(x) : x).} \tag{1}$$

The gas can lose at most as much entropy as was measured from it. And the budget is exactly achievable: if the demon spends its correlation completely and reversibly (so that $K(m(x), x) \stackrel{\pm}{=} K(m(x), y)$, no entropy produced, and $I(m(x) : y) \stackrel{\pm}{=} 0$, no correlation left unspent), then the displayed inequalities collapse to

$$K(x) - K(y) \stackrel{\pm}{=} I(m(x) : x).$$

Now place (1) beside Theorem B.2 (Appendix B) and read the two term by term. In each, the left side is the entropy reduction of the steered system, and the right side is the mutual information between the controller’s record (action, measurement) and the system’s state. The Touchette–Lloyd theorem is the ensemble (Shannon) version of the constraint: it speaks of distributions, averages, and a blind baseline supplied by the dynamics. Inequality (1) is the single-shot, individual-state (algorithmic) version: it speaks of the one actual gas state in front of the demon, with no ensemble anywhere, and the role of the blind baseline is played by the additive constant (which absorbs what the simply describable dynamics can do unaided). It is, simultaneously, the rigorous form of Type-3 optimization from Section 5: the copy step purchases $I(m(x) : x)$, the control step spends it, bit for bit.

One last reading of the demon’s situation will be the key to the final section. After the measurement, two distinct entropies attach to the same gas: its *objective* entropy $K(x)$, and its entropy *from the demon’s perspective*, the conditional complexity

$$K(x | m(x)) \stackrel{\pm}{=} K(x) - I(m(x) : x),$$

which is smaller, by exactly the measured information. The demon’s optimization power over the gas is precisely the wedge between the objective and the subjective entropy. Subjectivity has not been banished from thermodynamics; it has been *located*: the “subjective distribution” of the old formalism has become a physical conditioning on a physical record, carried in a memory that occupies physical space and obeys the second law.

8 Knowledge as a physical resource: optimization for embedded agents

This section pulls the previous ones together into a single picture: for an embedded agent, knowledge about the environment is a physical resource, and it is the same quantity as the agent’s capacity to optimize the environment.

8.1 From exogenous to endogenous knowledge

Compare the two frameworks.

In the Gibbs–Shannon framework, what an agent knows about a system is the distribution μ , and μ is supplied from outside the physics. Entropy, work capacity, and therefore optimization capacity are all defined relative to it. Two problems follow. If we change μ , the physics appears to change (the problem of Section 6.4); and μ is free, so knowledge can be smuggled in at no cost (the loophole of Section 6.3). At equilibrium neither problem bites, because there the right μ is forced and simple. They bite exactly when the agent’s knowledge is the interesting part.

For an embedded agent there is no such outside ledger. Its knowledge of the environment has to be stored physically, in a memory made of the same matter as the environment and obeying the same laws. Algorithmic thermodynamics lets us handle that knowledge directly. Write a for the physical state of the agent’s memory and s for the physical state of the environment. Then “what the agent knows about the environment” is the algorithmic mutual information

$$I(a : s) \stackrel{\pm}{=} K(s) - K(s | a),$$

the number of bits by which the agent’s memory shortens the description of the environment. No distribution appears. Knowledge has become an objective relation between two pieces of matter, as much a fact about the world as a distance or a voltage. Subjective beliefs are not thrown away; they are *implemented*: an agent whose memory holds a good model of the environment is one whose state has high $I(a : s)$, and $K(s | a)$ is the environment’s entropy *as that agent finds it* (the demon’s-eye view of Section 7.4).

Knowing more lets you optimize more. An agent that understands its environment better can do more to it. The framework makes this precise by joining two steps: from the agent’s beliefs to mutual information, and from mutual information to optimization.

Beliefs to mutual information. Suppose the agent’s memory a encodes a computable belief q about the environment (formally, a short fixed program reconstructs q from a). Given a , one way to describe the true state s is to recover q and write down the Shannon codeword of s under q , which has length about $\log \frac{1}{q(s)}$ bits (Section 2.3). The shortest description is no longer than this, so

$$K(s | a) \stackrel{+}{\leq} \log \frac{1}{q(s)}, \quad \text{and hence} \quad I(a : s) \stackrel{\pm}{=} K(s) - K(s | a) \stackrel{+}{\geq} K(s) - \log \frac{1}{q(s)}.$$

The more probability the belief puts on the true environment, the shorter that codeword, and the more mutual information the memory carries about the world.

Mutual information to optimization. The algorithmic Touchette–Lloyd bound (1) says the agent can remove at most $I(a : s)$ bits of complexity from the environment, and that this much is attainable. Putting the two steps together,

$$\text{optimization the agent can perform} \stackrel{+}{\geq} K(s) - \log \frac{1}{q(s)},$$

so the agent can drive the environment down to a residual complexity of about $\log \frac{1}{q(s)}$, which is its entropy as the agent’s belief sees it. *The more probability the agent’s model puts on the truth, the more of the environment it can optimize*, at a rate fixed by the bookkeeping rather than by any modeling choice.

This is just Bayesian learning, described physically. As the agent collects evidence and updates, $q(s)$ rises, the codeword $\log \frac{1}{q(s)}$ and the residual $K(s | a)$ shrink, $I(a : s)$ grows, and the optimization within reach grows with it. In the limit of certainty $K(s | a) \stackrel{\pm}{=} 0$ and $I(a : s) \stackrel{\pm}{=} K(s)$, and the agent can in principle steer the environment all the way down. Updating a belief toward the truth and building up spendable, physically stored knowledge are the same process. This is the single-state form of the Touchette–Lloyd result of Appendix B and the precise content of the Type-3 channel of Section 5.

Two further properties round out the picture of $I(a : s)$ as a resource.

- It is *used up* when spent. Optimization here is the Type-3 channel (Section 5): the shared correlation is consumed, so to keep going the agent must measure again (Type 2) or dump waste (Type 1).

- It *decays*. Correlations between two systems cannot grow without interaction (a consequence of the algorithmic second law, Section 7.3), so if the environment changes while the agent’s records stay fixed, $I(a : s)$ falls, and correlation that is allowed to decay can no longer be spent.

In short: an embedded agent’s knowledge about the world is the same physical quantity as its capacity to optimize the world beyond blind baselines. Agents that know more can do more, at a fixed exchange rate of one bit of optimization per bit of correlation.

8.2 The three types, sharpened by universal computation

With algorithmic entropy in hand, each of the three channels gets a refinement, and in every case the refinement turns on the compressibility of an individual state, which the ensemble account cannot express.

Type 1. Before dumping waste into the environment, an agent should ask whether the waste is as random as it looks. If it has compressible structure (and the output of any computation does, by Corollary 7.3), the agent can compress it first and export only $K(\text{waste})$ bits of real entropy instead of its raw size. This must happen *before* the waste mixes into the environment, because mixing destroys the structure: once it is gone, no compressor can recover it.

Type 2. The memory cost of recording a measured state x is not the raw length of the transcript but its complexity $K(x)$: records can be stored compressed, so a memory can absorb more optimization than its nominal size suggests. And $K(x)$ is a floor: no method stores x in fewer bits.

Type 3. The steering budget is the mutual information (1), and computation offers a second way to acquire it: if the environment’s state is simple ($K(s)$ small), a short program generates it, so the agent can come to know it (acquire $I(a : s) \approx K(s)$) by *computation alone*, with almost no measurement. Simple worlds are cheap to know and so cheap to optimize.

These three refinements meet in one fact, noted by Daniel C and Ebtekar: the complexity $K(s)$ of the optimized subsystem is at once the smallest memory that can absorb it (Type 2) and the smallest knowledge that can steer it (Type 3). What a state costs to record and what it takes to know are the same number of bits, because both are the length of its shortest description.

8.3 Dissolving the subjectivity

Return to the puzzle of Section 6.4: does the demon’s capacity to do work change when *our* beliefs about its memory change? The answer has two parts, and the second is the one that resolves it.

First, the demon’s capacity is objective. Its remaining capacity to optimize is its free memory: the memory’s size minus the *description complexity* of its current contents. If those contents are compressible, the demon can compress them (a reversible, free operation by Corollary 7.3) and recover the space; if they are incompressible, the space is irreducibly used, because freeing it would mean destroying information, which reversibility forbids. Either way the capacity depends only on the memory’s physical state. No one’s beliefs enter the formula.

Second, the appearance of belief-dependence was tracking something real. When we update our beliefs about the demon’s memory, that update is a physical change in our own brains, which now share more algorithmic mutual information with the demon’s memory. That shared information is itself a Type-3 resource: in principle we could spend it to help compress the demon’s memory and free capacity the demon alone could not reach. So the demon’s situation really is different after our update, but only relative to the larger system that now includes our correlated brains, and the difference is exactly the correlation we gained. What the old formalism recorded as a mysterious observer-dependence of entropy, the new one records as an ordinary physical relation between observer and observed.

8.4 Back to optimizers

Return to Section 3. An optimizer drives the system it acts on from a broad range of configurations into a narrow target set. What does the existence of such an optimizer imply, thermodynamically?

The funneling is coarse-grained entropy reduction in a subsystem, so the books must balance through some mixture of the three channels (Section 5): waste exported to the surroundings (Type 1), records kept in memory (Type 2), or pre-existing correlation spent (Type 3). The main implication is a selection-theorem-like conclusion: if the funneling beats the blind baseline of the environment’s own dynamics, then by the Touchette–Lloyd theorem (Theorem B.2, Appendix B) and its algorithmic form (1), the system must carry mutual information with what it steers, a necessary (though, by Appendix B.8, not sufficient) ingredient of a world model. So at least this much modeling is forced by the bookkeeping, not read in by an observer. This is the thermodynamic counterpart of the observer-independence argued in Section 3.5: the presence of a world model, like the presence of an optimizer, is an objective fact about the system, certified by the entropy it removes rather than attributed by whoever is watching.

9 Takeaways

- An optimizer is a physical thing whose presence drives the system it acts on, from a broad range of initial conditions and despite perturbations, into a narrow target: conditioning on the optimizer makes your uncertainty about where that system ends up small (it gives the system a convergent attractor). The amount of optimization is the entropy reduction it produces in that system, the bits by which it narrows the system’s possibilities. Calling something an optimizer requires no claim about its internals or goals, only about its effect on the system it steers.
- Even an observer who cares only about prediction has an objective, information-theoretic reason to single out optimizers: the target is a short, robust description of where the system ends up (it converges to the target), so it forecasts the macroscopic outcome for a tiny fraction of the bits that the initial-condition route would require, and without knowing the initial conditions at all. This makes optimization an observer-independent feature of the world, not a stance projected by a particular observer, and answers the worry that intelligence is merely task-relative.
- Microscopic physics is reversible, and the second law of thermodynamics follows from that reversibility once dynamics are coarse-grained: a doubly stochastic Markov step cannot decrease ensemble entropy (Theorem 4.1). Global entropy reduction is impossible; all optimization is local, and the books must balance elsewhere.
- Thermodynamic laws are about information, not molecules: they bind whenever a designer cannot observe a system, cannot destroy information, and faces a conservation constraint. In Wentworth’s coin world, work extraction is data compression, no work comes from a single maxentropic pool, and the fuel of every engine is negentropy, the gap between a system’s entropy and its constrained maximum.
- Steering beyond blindness costs information. The Touchette–Lloyd theorem (Appendix B) bounds any policy’s entropy reduction by the blind-dynamics baseline plus the mutual information between action and environment ($\Delta H \leq \Delta H_{\text{blind}}^{\text{max}} + I(X; A)$), so observed optimization beyond the baseline certifies the presence of modeling. The bound is a constraint, not a guarantee: information can be wasted.
- An embedded agent has exactly three ways to reduce a subsystem’s entropy: dump it into the environment as heat, absorb it into memory by measurement, or spend pre-existing mutual information with the subsystem. Maxwell’s demon is the second channel; its measure-then-steer operation decomposes into buying correlation and spending it.
- The Gibbs–Shannon entropy is defined relative to an exogenous, subjective ensemble, and this fails exactly on the states agents are made of: intricate knowledge gets booked at zero cost, non-concentrated ensembles yield averages true of no actual state, and atypical structure goes uncredited. Equilibrium thermodynamics is the special regime (simple, concentrated, typical ensembles) where the failure never bites.
- Algorithmic thermodynamics replaces the ensemble with Kolmogorov complexity: entropy becomes an objective property of the individual state, agreeing with the classical entropies in their domain and extending beyond it. The algorithmic second law holds per trajectory with exactly priced fluctuation

allowances ($K(t - s) + \log \frac{1}{\delta}$, covering Poincaré recurrence and chance dips); deterministic simple dynamics cannot create entropy; and the demon obeys $K(x) - K(y) \stackrel{+}{\leq} I(m(x) : x)$, the single-state form of Touchette–Lloyd. This objective entropy dissolves the subjectivity problem: it is a property of the state, not of any observer. Entropy must be uncomputable for any of this to work: a computable entropy admits a perpetual-motion machine.

- For embedded agents, knowledge is endogenous: an agent’s probabilistic model of its environment is physically encoded in memory, its content is the algorithmic mutual information between memory and world, and that mutual information is the consumable, perishable resource that funds optimization beyond blind baselines. Agents that know more about the world can do more to it, at a fixed exchange rate of one bit of optimization per bit of correlation.

A A thermodynamics of biased coins: the generalized heat engine

This appendix develops, as a self-contained worked example, the toy thermodynamic world referenced from two points elsewhere in these notes (the blind baseline of Appendix B and the Type-1 channel of Section 5). It can be read at any point after Section 4. It follows Wentworth’s essay *Generalized Heat Engine*, whose goal is to take the distinctively “thermo-flavored” ideas of statistical mechanics (heat, work, engines, the impossibility of perpetual motion) and rebuild them in a setting with no physics at all, so that it becomes clear which parts of thermodynamics are really facts about information. The model also makes vivid a viewpoint, the *designer’s viewpoint*, that transfers directly to thinking about agents embedded in environments they cannot fully observe.

A.1 The designer’s viewpoint

Why can’t a refrigerator designer simply build a machine that looks at the air molecules and removes the fast ones? Consider the designer’s epistemic situation. All the microscopic dynamics of the fridge-plus-environment system are reversible, so, as we established in Section 4.1, the number of possible microstates compatible with what is known never decreases on its own. The only way to reduce uncertainty about a microstate is to observe it. But the designer is designing the machine *in advance*: deciding today how the machine will behave, with no access to the exact positions the air molecules will occupy when the machine is eventually switched on. From the designer’s perspective there is uncertainty that cannot be reduced, only *moved around*. The machine itself can “observe” variables while running, but the machine is part of the total system, so its observations are just more reversible dynamics: any record the machine makes is a physical change in the machine, subject to the same conservation rules as everything else. (We will analyze exactly this maneuver, observation implemented as reversible copying, when we meet Maxwell’s demon in Section 5.)

The design problem, then, is to choose transformations, in advance and sight unseen, that leave us *more* certain about some chosen variables (the inside of the refrigerator) at the cost of becoming *less* certain about others (the heat baths that power the machine). Thermodynamic-style laws apply whenever three conditions hold: the designer cannot gain information about the system (no peeking at design time), information cannot be destroyed at the lowest level (reversibility), and there is some conserved quantity constraining the transformations (energy, in physics). Wentworth’s toy world has all three conditions, and nothing else.

A.2 The setup: coins, transformations, and two conservation laws

The world consists of two large pools of independent biased coins, where each coin shows 1 (heads) or 0 (tails):

- a *cold pool* $X_1^C, \dots, X_n^C$, in which each coin is heads with probability 0.1 (entropy $h(0.1) \approx 0.47$ bits per coin, by Example 2.2); and
- a *hot pool* $X_1^H, \dots, X_n^H$, in which each coin is heads with probability 0.2 (entropy $h(0.2) \approx 0.72$ bits per coin).

The heads-probability plays the role of temperature (the hot pool is more uncertain per coin), and the *number of heads* plays the role of energy. We may apply any transformation T that replaces the coins' values with new values computed from their old values, subject to three rules, mirroring the three conditions above:

1. **Reversibility.** T must be invertible: from the final state of all the coins (and knowledge of which transformation was applied) the initial state must be reconstructible. This is the analogue of microscopic reversibility.
2. **Conservation.** T must conserve the total number of heads, the analogue of energy conservation. Heads can be moved between coins but neither created nor destroyed on net.
3. **No peeking.** We choose T before looking at any coins, exactly as the refrigerator designer commits to a blueprint before the machine meets its environment.

To see that interesting transformations exist within these rules, here is Wentworth's example. Define T to act on three coins as follows: *if X_1^C shows tails, swap X_3^H with X_7^H ; if X_1^C shows heads, change nothing.* This conserves heads (it either swaps two coins, which permutes values without changing their sum, or does nothing), and it is reversible (the control coin X_1^C is itself unchanged, so from the final state we can tell whether the swap occurred, and applying the same transformation again undoes it). Conditional swaps like this, chained together, are the engine-builder's basic toolkit, and notice what the example already demonstrates: a transformation can *read* one part of the system and act on another, all within deterministic, reversible, conservative rules. Machines that "observe while running" are not outside the formalism; they are inside it.

In summary, we choose an invertible, heads-conserving map T from coin configurations to coin configurations, the world applies it, and we ask what such maps can accomplish.

A.3 Extracting work is a compression problem

Define a *work coin* to be a coin whose final value is heads with near-certainty (probability approaching 1 as n grows). Work coins are the analogue of stored mechanical work or a charged battery: a resource in a known, definite state, available to power other processes. *Extracting work* means choosing T so that some w designated coins end up as work coins.

The first key observation is that reversibility turns work extraction into a data compression problem. Write X for the (random) initial configuration of all the coins and $X' = T(X)$ for the final configuration. Because T is invertible, the final state determines the initial state: X' contains *all* of the information in X , so $H(X') \geq H(X)$ (in fact $H(X') = H(X)$, since invertible maps preserve entropy exactly). Now suppose w of the final coins are deterministic. Deterministic coins carry zero entropy, so all $H(X)$ bits of initial uncertainty must be carried by the remaining coins. In other words: *to extract w work coins, we must compress the information content of the entire initial state into the other coins.* Producing certainty in one place requires concentrating uncertainty elsewhere; nothing is ever erased.

A.4 No work from a single heat bath

Let us try to extract work from the hot pool alone, and watch the attempt fail; the failure is the toy-world version of the impossibility of a perpetual motion machine of the second kind (a machine that extracts work from a single-temperature heat bath).

The hot pool's n coins carry $H(X) = 0.72n$ bits of entropy. A compression scheme can in principle pack these bits into about $0.72n$ coins, leaving $0.28n$ coins deterministic. But here the conservation law steps in. Fully compressed data is incompressible, which means it looks statistically uniform: each compressed coin is heads with probability about $\frac{1}{2}$ (if the compressed coins were biased, they would be further compressible, contradicting full compression). So the $0.72n$ compressed coins would contain about $0.36n$ heads. The initial state only has $0.2n$ heads to go around, and heads are conserved. Even if every one of the would-be work coins were set to tails instead of heads, we cannot balance the books: the compression we need requires more heads than we possess. The attempt fails.

This argument generalizes well beyond coins, as follows. The hot pool’s distribution (independent coins, each heads with probability 0.2) is the *maximum-entropy* distribution among all distributions with its expected number of heads; in this precise sense a heat bath is “as random as its energy allows”. Suppose, in general, that a pool of variables is maxentropic subject to a constraint that fixes the value of some additive quantity $\sum_k f_k(X_k)$. If we deterministically fix the value of one variable (extract work from it), we shrink the set of values the constrained sum can take on the remaining variables, and therefore shrink the maximum entropy the remaining variables can hold. Since the initial state already saturated the maximum, the remaining variables cannot absorb all the information; compression must fail. *One cannot extract work from a system that is already at maximum entropy given its constraints.* The slack between a system’s actual entropy and its constrained maximum, which following standard usage we call *negentropy*, is the only fuel there is.

A.5 Work from two heat baths at different temperatures

Now use both pools: $2n$ coins, total entropy $\approx 0.47n + 0.72n = 1.19n$ bits, total heads $\approx (0.1 + 0.2)n = 0.3n$. The crucial difference from the single-pool case is that the *joint* initial distribution is not maxentropic given the joint constraint. The maximum-entropy distribution with $0.3n$ heads spread over $2n$ coins would make every coin heads with probability 0.15; our actual state instead maintains two distinct biases, 0.1 and 0.2. That gap between actual entropy and constrained maximum entropy is negentropy, and we can spend it.

Let us compute how much work it buys. Suppose we aim for w work coins (deterministic heads). The remaining $2n - w$ coins must carry all 1.19n bits of initial entropy while containing the remaining $0.3n - w$ heads. To carry as much information as possible, the final distribution of those $2n - w$ coins should itself be maxentropic subject to its heads constraint, which (for large n) means independent coins, each heads with probability

$$q = \frac{0.3n - w}{2n - w},$$

giving total entropy $(2n - w) h(q)$ where h is the binary entropy function of Example 2.2. The largest feasible w makes this capacity exactly equal to the required 1.19n bits:

$$(2n - w) h\left(\frac{0.3n - w}{2n - w}\right) = 1.19n.$$

Solving numerically gives $w \approx 0.011n$. So a heads-conserving, reversible, designed-in-advance transformation *can* extract work from two heat baths at different temperatures, converting about 3.7% of the total heads (5.5% of the hot pool’s heads) into work coins. This is the toy world’s heat engine, and we built it with nothing but information theory.

Wentworth notes one caveat about comparing this number to physics textbooks: the efficiency computed here is not the classical Carnot efficiency, because the classical result answers a slightly different question (the optimal conversion rate *at the margin*, for engines drawing from effectively inexhaustible baths, generally consuming unequal amounts from the hot and cold sides), whereas we asked how much total work can be extracted from two fixed, finite pools. The conceptual content (work comes only from temperature *differences*, never from a single bath) is the same.

A.6 What the toy world teaches

Four lessons from this construction recur throughout the rest of the notes, so we state them explicitly.

First, thermodynamic laws are not specifically about molecules and joules. They apply whenever an agent or designer cannot gain information about a system at will, cannot destroy information at the lowest level, and operates under some conservation constraint. This is why the same laws will reappear when we analyze agents, memories, and computations.

Second, under reversibility, uncertainty is never destroyed, only moved. Every machine that makes one variable more predictable makes others less predictable.

Third, extracting work (producing variables in definite, known states) is a compression problem, and conversely, compression capacity is work capacity. This identity between “useful energy” and “compressibility” is the seed of the algorithmic viewpoint in Section 7, where it will be sharpened from a statement about distributions into a statement about individual states.

Fourth, the fuel of all such machines is negentropy: the gap between a system’s entropy and the maximum entropy compatible with its constraints. A system at its constrained maximum entropy (a single heat bath) is useless as fuel, no matter how much “energy” it contains.

Everything in this section was achieved *blind*: the transformation was chosen before looking at any coins, and all the work came from statistical structure (the bias difference between pools) that was known at design time. The obvious next question is what an agent could additionally accomplish if it were allowed to *look*. That question has a precise answer, which we take up in Appendix B.

B Steering costs information: the Touchette–Lloyd theorem

B.1 From optimization to modeling: the question

In Section 5 we saw that an agent can steer a subsystem by spending mutual information it already shares with it (the Type-3 channel). This appendix quantifies that channel: how much entropy reduction does each bit of mutual information actually buy? The answer connects optimization to *modeling*.

One of the recurring hopes of agent foundations is to find *selection theorems*: results stating that any system selected to perform sufficiently well at some task must, as a matter of mathematical necessity, contain certain agent-like structures, such as a world model, a goal representation, or a planning process. The *agent structure problem* asks this in the converse direction to the usual one: instead of “agents bring about outcomes”, it asks “if we observe that something reliably brings about a particular outcome, can we conclude that it must be modeling its environment?” Wentworth posed a sharp version of the question: how many bits of optimization can one bit of observation unlock?

A theorem of Touchette and Lloyd, originally published in the control theory literature and brought to the attention of the agent foundations community by Harwood and Altair, is one of the few existing results that directly addresses this problem.³ Informally, the theorem says: *the entropy reduction a policy achieves, beyond what the environment’s dynamics would do on their own, is bounded by the mutual information between the policy’s action and the environment’s state*. Every bit of optimization beyond the blind baseline requires a bit of modeling. This appendix builds up the exact statement, with fully worked examples.

B.2 The setup: environments, actions, policies

The model is a single time step of an environment under influence. Three random variables appear:

- X , the *initial state* of the environment;
- A , the *action* taken (by an agent, a controller, a machine; the formalism does not care);
- Y , the *final state* of the environment.

Two conditional distributions specify the situation. The *policy* $P(A | X)$ describes how the action is chosen as a function of the environment’s initial state, and the *dynamics* $P(Y | X, A)$ describe how the final state is produced from the initial state together with the action. The dynamics are held fixed (they are the physics of the environment); the policy is the object of study. As is conventional, X and Y range over the same set of environment states, and the dynamics may be deterministic (all transition probabilities 0 or 1) or noisy.

Following Section 3.3, a policy is scored by the entropy reduction it achieves:

$$\Delta H := H(X) - H(Y),$$

³The result appears as Theorem 10 of H. Touchette and S. Lloyd, *Information-theoretic approach to the study of control systems*, Physica A 331:140–172 (2004); it is also described in their 2000 paper *Information-theoretic limits of control*, and an earlier, more physics-flavored proof appears in Lloyd’s 1989 work on the use of mutual information to decrease entropy, in the context of Maxwell’s demon. Related results connecting regulation to modeling include the good regulator theorem of Conant and Ashby and the internal model principle of control theory, but those concern conditions for *optimal* regulation, whereas the Touchette–Lloyd theorem is an inequality constraining *all* policies, optimal or not, which is what makes it feel like a genuine selection theorem.

the number of bits by which the final state is more predictable than the initial state was. We are measuring optimization exactly as Section 3 taught us to: by how much the process funnels a broad distribution into a narrow one. (And as noted in Remark 3.1, entropy reduction is the physics-facing half of expected utility maximization, the other half being which narrow region the agent prefers.)

B.3 Blind and sighted policies

Definition B.1 (Blind and sighted policies). A policy is *blind* if the action is statistically independent of the initial state, that is, if $P(A | X) = P(A)$, or equivalently $I(X; A) = 0$. A policy is *sighted* if $I(X; A) > 0$.

Blind policies include every deterministic rule of the form “always take action a_1 ”, and also every randomized rule whose randomness is independent of the environment, such as “flip a private coin; on heads do a_1 , on tails do a_2 ”. What blindness excludes is precisely any flow of information from the environment’s state into the choice of action. Sightedness is a matter of degree, and the degree is measured by $I(X; A)$: a policy that conditions on one observed bit of the state has $I(X; A) \leq 1$; a policy that observes everything can have $I(X; A)$ as large as $H(X)$. The whole transformation toolkit of the coin world (Appendix A) consisted of blind policies: the no-peeking rule was exactly the requirement $I(X; A) = 0$, with the “action” being the choice of transformation.

One might naively conjecture that any entropy reduction at all requires sight, but this is false, and seeing why sharpens the eventual theorem. The dynamics alone can reduce entropy: a contracting dynamics (one that funnels many initial states toward the same point on its own, like a ball settling to the bottom of a valley against friction) funnels states all by itself, whatever the action; and the coin engine of Appendix A.5 reduced the entropy of designated coins while completely blind. The right question is therefore not whether a policy reduces entropy, but whether it reduces entropy *more than blindness allows*. Accordingly, define the *blind baseline*

$$\Delta H_{\text{blind}}^{\max} := \max_{P(X) \in \mathcal{X}, P(A) \in \mathcal{A}} \Delta H,$$

the largest entropy reduction achievable by *any* blind policy from *any* initial distribution, where $\mathcal{X}$ is the set of all distributions over initial states and $\mathcal{A}$ is the set of all action distributions independent of X . This baseline is a property of the dynamics alone: it is everything the environment can be made to do “on its own”.

B.4 A worked example: the guessing game, played blind

The following game, from Harwood and Altair’s exposition, makes every quantity in the theorem concrete and computable.

A computer secretly picks a 5-bit binary string X , uniformly at random, so $H(X) = 5$ bits. You then submit a 5-bit string A of your own. The computer feeds both strings into the fixed, publicly known function

$$f(x, a) = \begin{cases} 00000 & \text{if } a = x \quad (\text{you matched the secret string}), \\ x & \text{otherwise,} \end{cases}$$

and outputs $Y = f(X, A)$. Your goal is to make the output distribution as predictable as possible, that is, to minimize $H(Y)$.

First, play blind: you must choose your string knowing nothing about X . How well can you do? Consider submitting a fixed string a . If you choose $a = 00000$, then matching produces output 00000 and failing to match reproduces X , which (conditional on not matching) is uniform over the 31 strings other than 00000; a short calculation shows the output is then exactly uniform over all 32 strings: $H(Y) = 5$ bits, no reduction at all. Choosing $a = 00000$ is the one genuinely wasted move.

Now choose any other fixed string, say $a = 11111$. Two initial states lead to the output 00000: the state $X = 11111$ (you matched, and the function outputs zeros) and the state $X = 00000$ (you did not match, and the function reproduces it). The output 11111 itself never occurs (if $X = 11111$ you matched it, producing zeros; otherwise the output is $X \neq 11111$). Every other string y occurs exactly when $X = y$. The output distribution is therefore

$$P(Y = 00000) = \frac{2}{32} = \frac{1}{16}, \quad P(Y = 11111) = 0, \quad P(Y = y) = \frac{1}{32} \quad \text{for the other 30 strings,}$$

with entropy

$$H(Y) = \frac{1}{16} \log 16 + 30 \cdot \frac{1}{32} \log 32 = 0.25 + 4.6875 \approx 4.94 \text{ bits.}$$

A modest improvement over 5 bits, but a real one: you have piled two initial states onto a single output, making the output distribution slightly lumpy and therefore slightly more predictable. By symmetry every fixed non-zero string performs identically, randomizing among them does no better, and one can check this is optimal among blind policies; for this game and this initial distribution, the blind optimum is $\Delta H \approx 0.06$ bits. (If the initial distribution over strings were non-uniform, the best blind strategy would be to guess the most probable string that is not 00000; the theorem’s baseline $\Delta H_{\text{blind}}^{\max}$ takes the maximum over initial distributions as well.)

B.5 The same game, played sighted

Now suppose that before you choose your string, you are shown the first k bits of the computer’s secret string. A natural strategy is to submit the string consisting of the k bits you saw, followed by all 1s (the continuation does not matter much, so long as you avoid completing the all-zeros string). Seeing k bits multiplies your chance of an exact match by 2^k , from $\frac{1}{32}$ to $\frac{1}{2^{5-k}}$, and each match funnels probability onto the single output 00000. Computing $H(Y)$ for each k exactly as above produces the following table.

bits of X observed	0	1	2	3	4	5
$H(Y)$ in bits	4.94	4.85	4.63	4.11	2.83	0
ΔH in bits	0.06	0.15	0.37	0.89	2.17	5

With all five bits observed you match the secret string every time, the output is deterministically 00000, and you achieve the maximum conceivable reduction $\Delta H = H(X) = 5$ bits. Each additional bit of observation buys additional steering power, and the more you know, the faster the returns in this particular game. Information about the environment converts into optimization of the environment.

B.6 Mutual information measures sightedness

To state the theorem we need to quantify “how sighted” a policy is, and the right quantity is exactly the mutual information of Definition 2.6. In the strategy above with $k = 2$, your action is determined by the two bits you observed (you always play “the two seen bits, then 11”). Imagine someone who knows your strategy and observes only your *action*, say $A = 10111$. They can immediately infer that the secret string begins with 10: the action reveals the observed bits. Before seeing your action their uncertainty about X was $H(X) = 5$ bits; after seeing it, $H(X | A) = 3$ bits; so $I(X; A) = H(X) - H(X | A) = 2$ bits, exactly the number of bits the policy “knows”. Mutual information formalizes the intuitive notion of how many bits of the environment’s state are reflected in the agent’s behavior, and it does so without any reference to the agent’s internals: it is a property of the joint statistics of state and action, estimable in principle by watching the system play many rounds.

B.7 The theorem

Theorem B.2 (Touchette–Lloyd). *Fix any dynamics $P(Y | X, A)$. Then for every initial distribution $P(X)$ and every policy $P(A | X)$,*

$$\Delta H \leq \Delta H_{\text{blind}}^{\max} + I(X; A).$$

In words: the entropy reduction achieved by an arbitrary policy decomposes into at most two budgets, namely everything the dynamics could have been steered to do by a blind controller, plus one bit for every bit of mutual information between the action and the environment’s initial state. The theorem holds with no assumptions about the policy’s internals, no optimality requirements, and no restriction on the dynamics. In particular, *it makes no reversibility assumption*: it is a purely information-theoretic (data-processing) inequality, valid for arbitrary dynamics, reversible or not. This is consistent with the rest of the development rather than in tension with it. The second law of Section 4.4 needed reversibility (in the form of double stochasticity) to forbid *global* entropy reduction; the Touchette–Lloyd theorem needs no such assumption

to bound the entropy reduction of a steered subsystem *beyond the blind baseline*. The two results constrain different quantities, and both hold at once.

The contrapositive direction gives the theorem its selection-theorem flavor. Suppose we observe a system achieving an entropy reduction ΔH strictly exceeding the blind baseline of its environment’s dynamics. Then we may *deduce*, sight unseen, that the system’s actions carry at least $\Delta H - \Delta H_{\text{blind}}^{\text{max}}$ bits of mutual information with the environment’s state. Mutual information with the environment is plausibly a necessary (though certainly not sufficient) ingredient of anything deserving the name *world model*; the theorem is thus a first rigorous step along the path “observed optimization implies internal modeling”, which is the path toward understanding when optimization implies agent-like structure. In the vocabulary of the guessing game: anyone who beats 4.94 bits of output entropy *must have peeked*, and the margin by which they beat it lower-bounds how much they saw.

B.8 What the theorem does not say

The inequality is an upper bound on what information makes possible, not a guarantee that information will be used well, and the gap matters in both directions.

First, mutual information can simply fail to help. If the dynamics ignore the action entirely (Y depends only on X), then every policy, however well-informed, performs exactly like a blind one: the channel from action to environment has zero capacity, and knowledge without influence is sterile.

Second, mutual information can be actively squandered. In the guessing game, consider the policy that observes the secret string completely and submits its bitwise negation (if the computer picked 01100, play 10011). This policy has the maximal $I(X; A) = 5$ bits of mutual information with the environment, and it *never* matches the secret string, so the output is always $Y = X$, uniform: $\Delta H = 0$, strictly worse than the optimal blind policy. Modeling the world is compatible with steering it badly.

So the theorem should be read as a conservation-style constraint, in the same family as the second law: it tells you what cannot happen (large optimization without modeling), never what must happen (modeling producing optimization). The analogy is exact in spirit, and Section 7 will turn it into a literal theorem of thermodynamics, where the role of $I(X; A)$ is played by the mutual information between a measurement record and the measured system.

Example B.3 (Noise-cancelling headphones). An everyday system displays the whole structure of the theorem. Let X be the ambient sound arriving at your ears, Y the sound you actually hear, and consider two technologies. Foam earplugs attenuate sound by passive damping. They are utterly blind ($I(X; A) = 0$; the plug’s “action” is the same whatever the sound), and yet they achieve a substantial entropy reduction: they exploit fixed statistical structure of the environment (sound is vibration, foam damps vibration), which is exactly a blind policy living off the $\Delta H_{\text{blind}}^{\text{max}}$ budget, like the coin engine living off the known bias difference. Active noise-cancelling headphones do something categorically different: they *listen* to the incoming sound and emit its inverted waveform. The emitted signal carries high mutual information with the ambient sound, and the entropy reduction correspondingly exceeds anything passive damping can achieve; the headphones can even steer selectively, cancelling the drone of an engine while passing a human voice. And playing music through the headphones is an entropy-*increasing* action, a reminder that agents are not obliged to minimize entropy; the theorem merely prices the steering they choose to do.

Remark B.4 (The coin engine, revisited). The generalized heat engine of Appendix A and the Touchette–Lloyd theorem are two halves of one picture. The engine shows what blind policies can extract: everything in the $\Delta H_{\text{blind}}^{\text{max}}$ budget, which is funded by pre-known statistical disequilibrium (the temperature difference between the pools). The theorem prices what blindness cannot reach: every further bit of entropy reduction costs a bit of mutual information acquired by observation. This is the quantitative form of the Type-3 channel of Section 5: $I(X; A)$ is precisely the correlation an agent spends when it steers, and Section 7.4 will show, with the books balancing exactly, what acquiring it costs in turn.

Sources and further reading

These notes integrate the following sources, listed in roughly the order their material appears.

- A. Flint, *The ground of optimization*, AI Alignment Forum, 2020. A complementary descriptive treatment of optimization as *optimizing systems* (a broad basin of attraction, a narrow target set, and robustness to perturbation), with a finer comparison of such systems along axes of robustness, duality, and retargetability. Recommended further reading on the behavioral characterization of Section 3.
- E. Yudkowsky, *Measuring optimization power*, LessWrong, 2008. A complementary quantitative proposal, measuring optimization by the improbability of the achieved outcome under random rearrangement; further reading alongside the entropy-reduction measure of Section 3.3.
- D. H. Wolpert and W. G. Macready, *No free lunch theorems for optimization*, IEEE Transactions on Evolutionary Computation 1(1):67–82, 1997; and D. C. Dennett, *The Intentional Stance*, MIT Press, 1987. The two articulations of the observer-relative view of intelligence and agency that Section 3.5 states and answers: no optimizer is universally competent (competence is relative to a problem class), and agency is a predictive stance an observer adopts rather than an intrinsic property.
- J. Wentworth, *Generalized heat engine*, LessWrong, 2020. The source for all of Appendix A: the designer’s viewpoint, the biased-coin world, work extraction as compression, and the two-bath engine. His decomposition of expected utility maximization (Remark 3.1) appears in *Utility maximization = description length minimization*, LessWrong, 2021.
- A. Harwood and A. Altair, *When bits of optimization imply bits of modeling: the Touchette–Lloyd theorem*, LessWrong, 2025. The pedagogical source for Appendix B, including the guessing game, the blind/sighted vocabulary, the headphone analogy, and the insufficiency caveats. The underlying theorem is from H. Touchette and S. Lloyd, *Information-theoretic approach to the study of control systems*, Physica A 331:140–172, 2004 (Theorem 10), with antecedents in their 2000 paper *Information-theoretic limits of control* and Lloyd’s 1989 work on Maxwell’s demon.
- Daniel C and A. Ebtekar, *Algorithmic thermodynamics and three types of optimization*, AI Alignment Forum, 2025. The central organizing source for these notes: the characterization of optimizers by the convergent attractors they create and the information-theoretic case for attending to them whatever one’s goals (Section 3), the three types of optimization (Section 5), the argument that entropy should objectively measure optimization capacity (Section 6), the algorithmic refinements of the three types, and the embedded-agency synthesis (Section 8).
- A. Ebtekar and M. Hutter, *Foundations of algorithmic thermodynamics*, Physical Review E, 2025 (arXiv:2308.06927). The formal backbone of Sections 4 and 7: Markovian coarse-grainings and the multibaker construction, Gács’ coarse-grained algorithmic entropy, Levin’s randomness conservation and the algorithmic second law with its $K(t - s) + \log \frac{1}{\delta}$ allowance, the exact Maxwell’s demon analysis including the partial-measurement bound (1), and the ensemble-versus-state comparison of Section 6 (including the robot-battery and bookshelf examples and Zurek’s identity).

For the broader agent foundations context assumed in Section 1 (true names and Goodhart’s law, reflective stability, embedded agency, selection theorems), see the Agent Foundations slide deck accompanying this module.